

INTERNSHIP PROJECT REPORT



Undertaken at

Vicharak Computers Private Limited

Surat, Gujarat – 395004, INDIA

Title

Peripheral Analyzer Sniffer

FPGA-Based Hardware Detection and Decoding of UART, I²C, and SPI Communication Protocols

Submitted by

Pranjal Upadhyay

Bachelor and Master of Technology (Dual Degree)

Department of Electronics and Communication Engineering

Indian Institute of Information Technology Design and Manufacturing, Kurnool

June 22, 2026

Abstract

This report presents the complete design, implementation, and experimental evaluation of the **Peripheral Analyzer Sniffer**, a self-contained, low-cost embedded hardware system designed for the autonomous detection and decoding of serial communication protocols used in modern microcontroller-based designs. The system targets three of the most widely deployed peripheral communication standards: the Universal Asynchronous Receiver-Transmitter (UART) protocol, the Inter-Integrated Circuit (I²C) protocol, and the Serial Peripheral Interface (SPI) protocol.

The hardware platform used is the **Vicharak Shrike Lite** development board, which co-integrates a **Renesas SLG47910 ForgeFPGA** and a **Raspberry Pi RP2040 dual-core ARM Cortex-M0+ microcontroller** on a single compact PCB. The **SLG47910 ForgeFPGA is the primary protocol analysis engine of the system**. It is responsible for asynchronous signal acquisition at its input pins, two-stage flip-flop synchronization for clock-domain crossing protection and metastability mitigation, hardware-based UART frame decoding encompassing start-bit detection, baud-rate-clocked midpoint sampling, and full byte extraction; hardware-based I²C frame decoding encompassing START and STOP condition detection, SCL-synchronized SDA bit sampling, 7-bit address extraction, data byte extraction, and ACK/NACK capture; and hardware-based SPI frame decoding encompassing chip-select edge detection, SCLK-synchronized bit shifting, simultaneous MOSI and MISO capture, and full-duplex byte reconstruction. All decoded events — whether originating from the UART, I²C, or SPI decoder — are packetized by a common event packet generator, buffered in an on-FPGA synchronous FIFO queue, and serialized through a dedicated host-interface block that streams structured event bytes to the RP2040 over the board's internal SPI bridge. **All protocol analysis, for every supported protocol, is performed entirely inside the Renesas SLG47910 ForgeFPGA.**

An ESP8266 Wi-Fi module is repurposed in this project as an external, deterministic test traffic generator. It produces UART, I²C, and SPI traffic for systematic functional evaluation of the FPGA decoder pipeline. The **RP2040 microcontroller operates strictly as an SPI host and display/GUI bridge**. Its sole firmware responsibilities are to perform packet acquisition of the pre-decoded packet stream arriving from the FPGA's host-interface port, parse the packet framing delimiters, format the extracted event fields into human-readable diagnostic messages, and forward this formatted output to a host PC over USB serial communication for display in a terminal or GUI application. The RP2040 SPI-to-USB Bridge Firmware performs no protocol detection and no protocol decoding of any kind.

The system demonstrates high-performance, real-time peripheral sniffing with minimal microcontroller involvement for protocol analysis. FPGA-based hardware acceleration ensures deterministic bit-timing precision and robust metastability mitigation, making the platform well-suited for hardware debugging, embedded-systems education, and low-cost diagnostic applications.

Keywords: Protocol Analyzer, ForgeFPGA, FPGA Hardware Acceleration, SPI, UART, I²C, Metastability, Two-Stage Synchronizer, FIFO, Serial Packetization, Embedded Systems, Verilog HDL, Hardware Decoder, Multi-Protocol Analyzer, Embedded Debugging, Logic Analyzer

Contents

List of Abbreviations	6
1 Introduction	7
1.1 Background and Motivation	7
1.2 Problem Statement	7
1.3 Scope of This Report	7
1.4 Document Organization	8
2 Objectives	8
3 Background Theory	9
3.1 UART Protocol — Foundations and Framing	9
3.1.1 UART Frame Structure	9
3.1.2 Baud Rate, Bit Period, and Midpoint Sampling	10
3.1.3 UART Configuration Parameters	10
3.1.4 Error Conditions in UART Communication	10
3.2 I ² C Protocol — Foundations and Framing	11
3.2.1 I ² C Bus Signals and Electrical Characteristics	11
3.2.2 START and STOP Conditions	11
3.2.3 Address Frame Structure	11
3.2.4 Data Byte Transfer	12
3.2.5 Complete I ² C Transaction Sequence	12
3.3 SPI Protocol Fundamentals	12
3.3.1 SPI Bus Signals	12
3.3.2 Full-Duplex Communication	13
3.3.3 Clock Polarity and Phase: SPI Modes	13
3.3.4 SPI Frame Structure	13
3.3.5 SPI Decoder Design Implications	14
3.4 Digital Metastability and Synchronizer Design	14
3.4.1 The Metastability Problem in Digital Circuits	14
3.4.2 Why Metastability is Unavoidable in This Project	14
3.4.3 Probabilistic Characterization: Mean Time Between Failures	14
3.4.4 Two-Stage Flip-Flop Synchronizer	15
3.4.5 Synthesis Constraint for Synchronizer Placement	15
4 Hardware Platform	15
4.1 Renesas SLG47910 ForgeFPGA — Primary Protocol Analysis Engine	15
4.2 RP2040 Microcontroller — SPI-to-USB Communication Bridge	16
4.3 ESP8266 NodeMCU — Test Traffic Generator	17
4.4 Complete Hardware Components Reference	17
5 Software Tools and Development Environment	18
5.1 Go Configure FPGA Design Suite (Renesas)	18
5.2 Arduino IDE with arduino-pico Core	18
5.3 Thonny IDE with MicroPython (ESP8266)	18
5.4 Serial Terminal (PuTTY)	18
5.5 GitHub for Version Control and Collaboration	18
6 System Architecture	18
6.1 Architectural Overview and Signal Flow	18
6.2 Hardware-Software Partitioning Rationale	19
6.3 SPI Target Interface and Pin Routing	19
7 FPGA Design	20
7.1 Overview of the Complete FPGA Design	20
7.2 Top-Level FPGA Module	20

7.2.1	Purpose and Architecture	20
7.2.2	Verilog Source Code	20
7.2.3	Detailed Code Explanation	24
7.3	UART Receiver Core Block	25
7.3.1	Operational Overview	25
7.3.2	Verilog Source Code	25
7.3.3	Explanation of the source code	26
7.4	I ² C Decoder Block	29
7.4.1	Operational Overview	29
7.4.2	Verilog Source Code	29
7.4.3	Explanation of the source code	31
7.5	SPI Decoder Block	33
7.5.1	Operational Overview	33
7.5.2	SPI Decoder Module (<code>spi_decoder.v</code>)	33
7.5.3	Detailed Code Explanation	34
7.6	UART FIFO Buffer	35
7.6.1	Purpose and Architecture	35
7.6.2	Verilog Source Code	35
7.6.3	Detailed Code Explanation	36
7.7	SPI Streaming Interface Block	37
7.7.1	Operational Overview	37
7.7.2	Verilog Source Code	38
7.7.3	Detailed Code Explanation	40
8	Firmware Architecture (Arduino MCU Code)	40
8.1	Microcontroller Role and Boundaries	40
8.2	Firmware Implementation	41
8.2.1	UART Signal Generator	41
8.2.2	I ² C Signal Generator	41
8.2.3	SPI Signal Generator	42
8.3	Detailed Logic Explanation	42
8.3.1	UART Signal Generator Explanation	42
8.3.2	I ² C Signal Generator Explanation	43
8.3.3	SPI Signal Generator Explanation	43
9	RP2040 MicroPython Host Reader	43
9.1	Role of the RP2040 in the Final System	43
9.2	Complete MicroPython Source Code	43
9.3	Detailed Code Explanation	46
10	Hardware Interfacing and Pin Configurations	46
10.1	Go Configure IO Planner Mapping	46
10.2	Physical Hardware Connections	47
11	Host Graphical User Interface (GUI)	47
11.1	Using the PyQt6 Host GUI	47
11.1.1	Download and Installation	47
11.1.2	Connecting to the Analyzer	48
11.2	Alternative: Thonny Shell Monitoring	48
12	Experimental Results	49
12.1	UART Decoding Results	49
12.1.1	Example: User-Generated UART Message	49
12.2	I ² C Decoding Results	50
12.2.1	Example: I ² C Message Transmission	50
12.3	SPI Decoding Results	52
12.3.1	Decoded SPI Output	52

12.4	Metastability at the FPGA Input Boundary	53
12.5	Simultaneous FIFO Write Arbitration	53
12.6	I ² C Clock Stretching Compatibility	53
12.7	SPI MISO Output-Enable Configuration	54
12.8	SPI Clock Domain Crossing Synchronization	54
13	Future Scope and Roadmap	54
13.1	Near-Term FPGA Protocol Additions	54
13.1.1	CAN Decoder	54
13.1.2	LIN Decoder	54
13.1.3	Automatic UART Baud Rate Detection	54
13.1.4	1-Wire Protocol Decoder	55
13.2	FPGA Architecture Improvements	55
13.3	Host-Side and Firmware Improvements	55
14	Conclusion	56
	Project Resources	56

List of Figures

1	UART Frame for ASCII Character ‘A’ ($0x41 = 0100\ 0001_2$) at 8N1 Configuration. LSB-first transmission: D0=1, D1=0, D2=0, D3=0, D4=0, D5=0, D6=1, D7=0.	9
2	Main window of the PyQt6 Host GUI	48
3	UART Example: FPGA-decoded UART messages displayed on the Host GUI.	49
4	Hardware-level UART frame capture confirming signal integrity and timing.	50
5	I ² C Example: FPGA-decoded I ² C transactions displayed on the Host GUI (Part 1). . .	51
6	I ² C Example: FPGA-decoded I ² C transactions displayed on the Host GUI (Part 2). . .	51
7	Hardware-level I ² C frame capture confirming signal integrity and timing.	52
8	SPI Example: FPGA-decoded SPI transactions displayed on the Host GUI.	53

List of Tables

2	UART Configuration Parameters	10
3	SPI Clock Modes Defined by CPOL and CPHA	13
4	Complete Hardware Component List	17
5	Go Configure FPGA I/O Planner Pin Mapping	47
6	Physical Hardware Connections Table	47

List of Abbreviations

S. No.	Abbreviation	Full Form
1	ACK	Acknowledgement
2	ASCII	American Standard Code for Information Interchange
3	BIT_PERIOD	Clock Cycles per UART Bit Period
4	BRAM	Block Random Access Memory
5	CAN	Controller Area Network
6	CDC	Clock Domain Crossing
7	CLB	Configurable Logic Block
8	CLK	Clock
9	CPHA	Clock Phase (SPI Parameter)
10	CPOL	Clock Polarity (SPI Parameter)
11	CS	Chip Select
12	EDA	Electronic Design Automation
13	FIFO	First-In, First-Out
18	FPGA	Field-Programmable Gate Array
19	FSM	Finite State Machine
20	GND	Ground
21	GPIO	General-Purpose Input/Output
22	GUI	Graphical User Interface
23	HDL	Hardware Description Language
24	I ² C	Inter-Integrated Circuit
25	LIN	Local Interconnect Network
26	LSB	Least Significant Bit
27	MCU	Microcontroller Unit
28	MISO	Master-In Slave-Out
29	MOSI	Master-Out Slave-In
30	MSB	Most Significant Bit
31	NACK	Negative Acknowledgement
32	PCB	Printed Circuit Board
33	RP2040	Raspberry Pi RP2040 Microcontroller
34	RTL	Register-Transfer Level
35	RX	Receive
36	SCL	Serial Clock Line (I ² C)
37	SCLK	Serial Clock (SPI)
38	SDA	Serial Data Line (I ² C)
39	SPI	Serial Peripheral Interface
40	SS	Slave Select
41	TX	Transmit
42	UART	Universal Asynchronous Receiver-Transmitter
43	USB	Universal Serial Bus
44	USB CDC	USB Communications Device Class
45	VCC	Voltage Common Collector

1 Introduction

1.1 Background and Motivation

Modern embedded systems are architecturally complex assemblies of heterogeneous subsystems that communicate using a variety of well-defined serial protocols. A single-board computer, automotive Engine Control Unit (ECU), or industrial IoT sensor node may simultaneously employ UART links for host terminal communication, I²C buses for sensor and peripheral interfacing, SPI buses for high-speed memory and display access, and CAN buses for fault-tolerant control networks. At each development milestone—from initial hardware bring-up and firmware debugging to final system validation and reverse engineering of third-party hardware—the ability to passively observe, capture, and correctly interpret these communication streams is fundamental.

Commercial protocol analyzers and mixed-signal oscilloscopes (for example, the Saleae Logic Pro 16, the Keysight MSOX series, and the Digilent Digital Discovery) offer comprehensive protocol decode feature sets but are sold at prices ranging from several hundred to several thousand US dollars. This price point is impractical for individual engineering students, small-scale academic research laboratories, and budget-constrained development projects. Open-source software solutions such as PulseView paired with the Sigrok framework offer a partial alternative but still require a compatible USB logic-analyzer frontend that demands additional hardware procurement and driver configuration.

This project directly addresses this accessibility gap by constructing a complete, self-contained, FPGA-based Peripheral Analyzer Sniffer from commodity development hardware. The solution leverages the co-integration of an FPGA and a microcontroller on the Vicharak Shrike Lite board to create a system in which the **FPGA serves as the primary hardware-based protocol analysis engine**—performing every stage of signal synchronization, protocol detection, and frame decoding for UART, I²C, and SPI in dedicated combinational and sequential hardware logic—while the microcontroller acts purely as a USB serial display and GUI communication bridge, without participating in protocol decoding. The resulting architecture supports concurrent monitoring and decoding of UART, I²C, and SPI traffic, allowing a single compact board to replace what would otherwise require three separate single-protocol analyzers.

1.2 Problem Statement

The fundamental challenge addressed by this project is formally stated as follows: given raw asynchronous and synchronous digital signals arriving at FPGA hardware input pins from an external microcontroller operating in a completely unrelated clock domain, fully decode the UART, I²C, and SPI protocol frames carried by those signals in real time using *hardware logic alone*, without delegating any aspect of protocol analysis to the host microcontroller.

This general problem decomposes into the following engineering sub-challenges:

- **Clock-Domain Crossing Safety:** Managing the boundary between asynchronous external signals and the FPGA system clock without introducing metastability-induced data corruption into downstream decoder logic.
- **Resource-Efficient State Machine Design:** Implementing correct and complete hardware state machines for UART, I²C, and SPI protocol decoding within the limited configurable logic capacity of the Renesas SLG47910 ForgeFPGA.
- **SPI Frame Reconstruction:** Implementing a hardware SPI decoder capable of detecting chip-select assertion, synchronizing to an externally generated SCLK, and reconstructing full-duplex MOSI and MISO byte streams, capable of supporting configurable CPOL and CPHA settings.
- **Buffered Event Management:** Implementing a reliable synchronous FIFO buffer on the FPGA to decouple the real-time decoding rate from the packet serialization rate, preventing event loss during periods of dense protocol traffic across all three monitored protocols.
- **Serialized Packet Streaming:** Designing a structured, framed packet format and a corresponding serializer block on the FPGA to reliably stream all decoded events — UART, I²C, and SPI alike — across the internal FPGA-to-MCU bridge interface.
- **Lightweight Bridge Firmware:** Developing RP2040 SPI-to-USB Bridge Firmware that only parses and formats pre-decoded FPGA event packets over USB, with no protocol intelligence of its own, so that the microcontroller imposes no timing constraints on the FPGA analysis pipeline.

1.3 Scope of This Report

This report provides a comprehensive, fully detailed technical description of every component of the Peripheral Analyzer Sniffer system. The scope encompasses:

1. A precise description of the hardware platform and each component's relevant electrical and logical characteristics.
2. A thorough theoretical treatment of UART, I²C, and SPI communication protocols, including frame structure, timing requirements, and error conditions.
3. A rigorous treatment of digital metastability theory, its probabilistic characterization, and the classical two-stage synchronizer solution.
4. A complete and deeply annotated description of the FPGA RTL design, covering the synchronization layer, UART decoder, I²C decoder, SPI decoder architecture, FIFO event buffer, and host-interface serializer.
5. A detailed analysis of the RP2040 SPI bridge firmware, with line-level explanation of every code block.
6. Experimental results from UART, I²C, and SPI decoding validation tests, including SPI loopback experiments, with quantified performance metrics.
7. A forward-looking roadmap for future enhancements and protocol expansion.

1.4 Document Organization

The remainder of this report is organized as follows. Section 2 states the formal project objectives. Section 3 develops the background theory required to understand every design decision, including SPI protocol fundamentals. Section 4 describes the hardware platform. Section 5 lists the software tools and development environment. Section 6 presents the complete system architecture and partitioning rationale. Section 7 gives the full FPGA design with annotated code, including the SPI decoder block. Section 8 presents the firmware architecture for the MCU traffic generator. Section 9 details the physical hardware connections and pin routing. Section 10 reports experimental results and functional validation, including SPI loopback testing. Section 11 outlines the future development scope and roadmap. Section 12 concludes the report.

2 Objectives

The primary objectives of this project, as scoped and executed, are enumerated below. Each objective is framed as a verifiable engineering deliverable.

- O1.** Design and implement a cost-effective, portable, self-contained FPGA-based protocol analyzer using the Vicharak Shrike Lite development platform, targeting a low-cost implementation using readily available hardware components.
- O2.** Implement asynchronous signal acquisition with two-stage flip-flop synchronization on the ForgeFPGA for all monitored input signals, in order to eliminate metastability hazards at the asynchronous clock-domain boundary before any decoder logic operates on the signals.
- O3.** Implement a fully functional hardware UART decoder on the ForgeFPGA in Verilog HDL that enables real-time start-bit detection, baud-rate clock division, midpoint bit sampling, and 8-bit byte extraction, operating at a standard 9600 baud target rate.
- O4.** Implement a dedicated hardware I²C decoder on the ForgeFPGA in Verilog HDL that enables real-time START condition detection, STOP condition detection, SCL-synchronized SDA bit sampling, 7-bit slave address extraction, data-byte extraction, and ACK/NACK capture.
- O5.** Implement a comprehensive SPI protocol decoder in Verilog HDL capable of monitoring MOSI, MISO, SCLK, and CS signals and reconstructing SPI frames in hardware.
- O6.** Implement an FPGA-based synchronous circular FIFO event buffer and a host-interface serializer block that packetize all decoded events — from the UART, I²C, and SPI decoders — into a structured, framed packet format and stream them reliably to the RP2040 over the board's internal SPI port.
- O7.** Develop a lightweight SPI-to-USB Bridge Firmware on the RP2040 that receives the FPGA-decoded packet stream via hardware SPI, parses the packet framing, formats the event payload into human-readable diagnostic text, and transmits that text to a host PC via USB serial communication for terminal or GUI display. The RP2040 SPI-to-USB Bridge Firmware shall contain *no* protocol detection or decoding logic of any kind.

- O8.** Design an extensible FPGA top-level architecture that supports future protocol expansion—for example, CAN and 1-Wire decoders—through incremental RTL block additions without modification of the RP2040 SPI host reader bridge firmware or packet transport layer.

3 Background Theory

This section provides the theoretical foundations necessary to understand every design decision made in this project. It covers UART protocol theory, I²C protocol theory, SPI protocol theory, and digital metastability theory in sufficient depth that a reader with a basic digital-electronics background can follow all subsequent design sections without external reference.

3.1 UART Protocol — Foundations and Framing

Universal Asynchronous Receiver-Transmitter (UART) is one of the oldest, simplest, and most widely used serial communication standards in digital systems. Its origins trace to the era of TTY teletypewriter equipment in the 1960s, and despite its age, it remains the default choice for host terminal communication, bootloader programming interfaces, GPS module output, and debug logging in virtually every class of microcontroller-based product.

The defining characteristic of UART is that it is *asynchronous*: there is no shared clock signal between transmitter and receiver. Unlike SPI or I²C, which distribute a master-generated clock to all participants, a UART communication link relies entirely on both parties having been pre-configured to the same **baud rate** — the number of symbol transitions per second. Without this pre-agreed rate, the receiver has no reference for when each bit begins and ends, and data recovery is impossible.

3.1.1 UART Frame Structure

A single UART data frame transmitting one byte consists of the following fields, transmitted serially on a single wire, LSB first:

1. **Idle State:** When no data transmission is in progress, the UART signal line is held at logical HIGH (the *mark* state, typically equal to the supply voltage VCC). This establishes a known baseline that allows the receiver to unambiguously identify the start of a new frame.
2. **Start Bit:** To signal the beginning of a frame, the transmitter drives the line LOW for exactly one full bit period ($T_{bit} = 1/\text{baud rate}$). This HIGH-to-LOW transition is the synchronization edge that the receiver uses to align its internal bit clock. Without this edge, the receiver would have no way to determine when the data bits begin.
3. **Data Bits:** Between 5 and 9 data bits follow the start bit. The most common configuration by far is 8 data bits. Data bits are transmitted LSB-first: the least significant bit (bit 0) is transmitted first, followed by bits 1 through 7.
4. **Parity Bit (optional):** An optional single parity bit may be inserted immediately after the last data bit. Even parity sets this bit such that the total number of logical 1s in the data plus parity field is even; odd parity ensures it is odd. Parity allows detection of any single-bit transmission error. Many modern UART links omit parity for simplicity (the “N” in “8N1” denotes no parity).
5. **Stop Bit(s):** One or two stop bits, held at logical HIGH, terminate the frame. The stop bit serves two purposes: it confirms that the line has returned to the idle HIGH state before the next frame, and it provides a mandatory inter-frame dead time that allows the receiver to complete processing of the current byte before the next start bit can be detected.

The shorthand “8N1” is universally used to describe the most common UART configuration: 8 data bits, No parity, 1 stop bit. This project targets 8N1 exclusively.

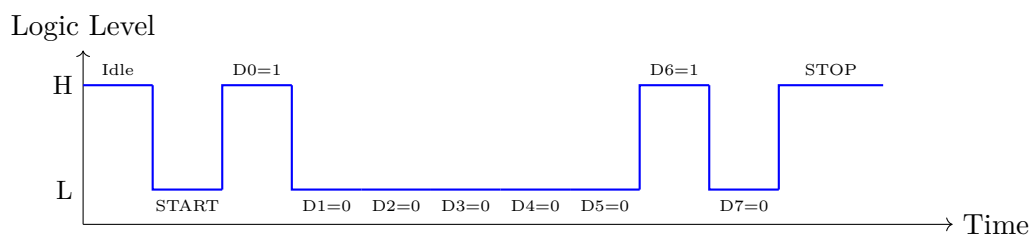


Figure 1: UART Frame for ASCII Character ‘A’ (0x41 = 0100 0001₂) at 8N1 Configuration. LSB-first transmission: D0=1, D1=0, D2=0, D3=0, D4=0, D5=0, D6=1, D7=0.

3.1.2 Baud Rate, Bit Period, and Midpoint Sampling

For a configured baud rate of B symbols per second, the bit period — the duration of each transmitted bit — is:

$$T_{bit} = \frac{1}{B} \quad (\text{seconds})$$

For the standard 9600 baud configuration:

$$T_{bit} = \frac{1}{9600} \approx 104.17 \mu s$$

Because UART has no transmitted clock, the receiver must independently measure elapsed time from the detected start-bit falling edge to determine when each data bit is at its stable centre. The fundamental technique for this is called **midpoint sampling**: the receiver waits exactly $T_{bit}/2$ after detecting the start bit falling edge before taking the first sample. This places the first sample at the centre of the start bit, and all subsequent samples at T_{bit} intervals thereafter.

Midpoint sampling is critical because it maximises the timing margin available before and after each sample point. Sampling at the exact centre of a bit period means that up to $\pm T_{bit}/2$ of accumulated timing jitter or clock-rate mismatch can be tolerated before a sample falls outside its correct bit interval. In contrast, sampling at the beginning or end of a bit period would provide zero margin and guarantee bit errors under any real-world jitter.

In the FPGA hardware decoder, midpoint sampling is implemented by loading a counter with `HALF_PERIOD` clock cycles on start-bit detection, then reloading with `BIT_PERIOD` clock cycles between each subsequent sample. At a 10 MHz system clock and 9600 baud:

$$\text{BIT_PERIOD} = \left\lfloor \frac{10,000,000}{9600} \right\rfloor = 1042 \text{ clock cycles}$$

$$\text{HALF_PERIOD} = \left\lfloor \frac{1042}{2} \right\rfloor = 521 \text{ clock cycles}$$

3.1.3 UART Configuration Parameters

The complete set of UART configuration parameters that must match between transmitter and receiver is:

S.No.	Parameter	Common Values	This Project
1	Baud rate	300, 1200, 9600, 115200, 921600	9600
2	Data bits	5, 6, 7, 8, 9	8
3	Parity	None (N), Even (E), Odd (O)	None
4	Stop bits	1, 1.5, 2	1

Table 2: UART Configuration Parameters

3.1.4 Error Conditions in UART Communication

A robust UART decoder must be able to identify and handle the following error conditions:

- **Framing Error:** When the receiver samples the stop bit position and finds a logical LOW instead of the expected HIGH, it indicates that synchronization has been lost. This typically occurs when the transmitter and receiver baud rates differ significantly, or when line noise corrupts the start-bit edge timing. In this project, framing errors are detected when `rx_sync` is LOW during the stop-bit sampling window; the affected byte is silently discarded.
- **Parity Error:** The received parity bit does not match the computed parity of the data byte. Not applicable in 8N1 mode.
- **Overflow Error:** A new frame begins arriving before the receiver has consumed the previous decoded byte. In a hardware FIFO implementation, this manifests as a FIFO full condition during a write attempt.
- **Break Condition:** The UART line is held continuously LOW for longer than one full frame period. This violates the stop-bit rule and is used by some systems as an out-of-band reset or attention signal. The current implementation does not explicitly detect break conditions.

- **Noise Glitch on Start Bit:** A very short LOW pulse on the idle line that is too brief to be a genuine start bit. The FPGA decoder's HALF_PERIOD verification step (described in detail in Section 7) filters these glitches by confirming the line is still LOW at the mid-start-bit sampling point.

3.2 I²C Protocol — Foundations and Framing

Inter-Integrated Circuit (I²C, often written I2C) is a synchronous, multi-master, multi-slave, half-duplex two-wire serial bus protocol originally developed by Philips Semiconductors (now NXP) in 1982. It has since become the dominant standard for short-distance, low-to-medium-speed peripheral interfacing in embedded systems, used universally for temperature sensors, EEPROM memories, real-time clock modules, display controllers, inertial measurement units, and hundreds of other device categories.

Unlike UART, which requires a dedicated point-to-point wire pair for each device, I²C implements a shared bus: all devices on the bus, whether masters or slaves, share exactly two signal lines. This makes it possible to connect dozens of peripherals to a microcontroller using only two GPIO pins.

3.2.1 I²C Bus Signals and Electrical Characteristics

I²C uses exactly two signal lines:

- **SCL (Serial Clock Line):** Driven by the bus master to provide a reference clock that synchronizes all bit transfers. In a multi-master environment, any master may drive SCL. Slaves may hold SCL LOW to implement *clock stretching*, which pauses the transaction while the slave prepares data.
- **SDA (Serial Data Line):** Carries address and data bits bidirectionally between master and slaves. Both master and slave may drive SDA, but only the master determines when transmissions occur. The fundamental rule is that SDA is stable and valid for sampling only when SCL is HIGH; SDA transitions are permitted only when SCL is LOW, except for the deliberately special START and STOP conditions.

Both SCL and SDA use **open-drain (wired-AND) topology**. Each device drives the line LOW by asserting its open-drain transistor output, but no device actively drives the line HIGH — instead, external pull-up resistors (typically 4.7 kΩ for standard mode) restore the line to HIGH when no device is driving it LOW. This topology allows multiple devices to share the bus safely without bus contention, and it is why I²C requires external pull-up resistors.

3.2.2 START and STOP Conditions

The I²C protocol reserves two specific signal patterns as special bus conditions. These patterns deliberately violate the normal data-valid rule (SDA stable while SCL is HIGH) and therefore cannot be confused with ordinary data:

- **START Condition:** A HIGH-to-LOW transition on SDA *while* SCL is HIGH. Only the bus master is permitted to generate a START condition. It uniquely signals the beginning of a new transaction and claims ownership of the bus.
- **STOP Condition:** A LOW-to-HIGH transition on SDA while SCL is HIGH. Generated by the master to signal the end of a transaction and release the bus, allowing other masters or future transactions to begin.
- **Repeated START (Sr):** A START condition generated without a preceding STOP condition. Used by the master to change direction (from write to read) or address a different slave mid-transaction without releasing the bus.

The FPGA I²C decoder detects START and STOP conditions using synchronous edge detection registers, comparing the current and previous clock-cycle values of the synchronized SDA signal against the current value of the synchronized SCL signal. This is described in full detail in Section 7.

3.2.3 Address Frame Structure

Following a START condition, the master must identify which slave it wishes to communicate with. The master transmits an **address frame** consisting of:

- **Bits 7 to 1 (7 bits):** The 7-bit device address, transmitted MSB-first. This provides $2^7 = 128$ unique device addresses, though addresses 0x00–0x07 and 0x78–0x7F are reserved by the I²C specification for special purposes, leaving 112 usable addresses for general devices.

- **Bit 0 (1 bit):** The $R/\bar{W}$ (read/not-write) direction bit. A value of 0 indicates that the master intends to *write* data to the addressed slave. A value of 1 indicates that the master intends to *read* data from the addressed slave.
- **9th Clock Pulse — ACK/NACK:** After the master has transmitted all 8 bits of the address frame, the master releases SDA (allowing the pull-up to pull it HIGH) and pulses SCL a 9th time. If the addressed slave is present and ready, it pulls SDA LOW during this 9th clock pulse, producing an **Acknowledge (ACK)**. If no slave recognizes the address, SDA remains HIGH (the pull-up state), producing a **Not-Acknowledge (NACK)**.

3.2.4 Data Byte Transfer

After the address frame and its ACK, data bytes are transferred. Each byte follows the same pattern: 8 data bits, transmitted MSB-first, followed by a 9th clock pulse during which the *receiver* signals ACK or NACK. In a write transaction, the master transmits and the slave acknowledges. In a read transaction, the slave transmits and the master acknowledges (with a NACK on the final byte to signal the end of the read).

3.2.5 Complete I²C Transaction Sequence

A complete I²C write-then-read transaction (as generated by the ESP8266 in this project's evaluation tests) follows this precise sequence:

1. Bus is idle: both SDA and SCL HIGH.
2. Master generates START: SDA falls while SCL HIGH.
3. Master clocks out 7-bit address, MSB first, followed by $R/\bar{W} = 0$.
4. Master releases SDA; slave pulls SDA LOW during 9th SCL pulse (ACK).
5. Master clocks out 8-bit register address byte.
6. Slave ACKs.
7. Master generates STOP (or Repeated START for read phase).
8. Master generates START again.
9. Master clocks out 7-bit address followed by $R/\bar{W} = 1$.
10. Slave ACKs.
11. Slave drives SDA and clocks out data byte(s), MSB first.
12. Master ACKs each byte; master NACKs the final byte to end the read.
13. Master generates STOP. Bus returns to idle.

3.3 SPI Protocol Fundamentals

The Serial Peripheral Interface (SPI) is a synchronous, full-duplex, master-slave serial communication standard originally introduced by Motorola in the 1980s. SPI is distinguished from both UART and I²C by its use of a dedicated, master-generated clock line and dedicated unidirectional data lines for each direction, which together permit simultaneous bidirectional data transfer at significantly higher clock rates than I²C. SPI is the dominant interface for high-speed peripherals such as SD cards, SPI flash memory, TFT/OLED displays, ADCs, and DACs.

3.3.1 SPI Bus Signals

A standard single-master, single-slave SPI link uses four signal lines:

- **SCLK (Serial Clock):** Generated exclusively by the master. Every bit transfer, in both directions, is referenced to an edge of this clock. Unlike I²C, SPI slaves never generate or stretch the clock.
- **MOSI (Master-Out, Slave-In):** A unidirectional line carrying data from the master to the slave. Driven exclusively by the master.
- **MISO (Master-In, Slave-Out):** A unidirectional line carrying data from the slave to the master. Driven exclusively by the selected slave; when not selected, a slave's MISO output must remain high-impedance to avoid bus contention with other slaves on a shared bus.

- **CS / SS (Chip Select / Slave Select):** An active-low control line, individually routed from the master to each slave device. The master asserts CS LOW immediately before beginning a transaction and de-asserts it HIGH immediately after, framing the entire transaction. Unlike UART and I²C, SPI has no in-band START or STOP symbol; the CS line itself performs this framing function.

3.3.2 Full-Duplex Communication

A defining characteristic of SPI is that it is **full-duplex**: on every single SCLK edge, one bit is simultaneously shifted out on MOSI *and* one bit is simultaneously shifted in on MISO. There is no separate read or write phase as in I²C; both shift registers (master and slave) operate in lock-step, exchanging data in both directions within the same clock cycle. This property makes SPI exceptionally well suited to applications that require continuous high-throughput data exchange, such as streaming ADC samples or driving a display frame buffer.

3.3.3 Clock Polarity and Phase: SPI Modes

SPI defines two configuration bits, **CPOL** (Clock Polarity) and **CPHA** (Clock Phase), which together determine exactly when data is considered valid and when it is sampled. These two bits define four standard SPI modes:

- **CPOL** determines the idle (inactive) level of SCLK. CPOL = 0 means SCLK idles LOW; CPOL = 1 means SCLK idles HIGH.
- **CPHA** determines which SCLK edge is used to sample data. CPHA = 0 means data is sampled on the *first* clock edge after CS assertion (and shifted on the second); CPHA = 1 means data is sampled on the *second* edge (and shifted on the first).

S.No.	Mode	CPOL	CPHA	Sampling Edge
1	0	0	0	Rising edge, SCLK idles LOW
2	1	0	1	Falling edge, SCLK idles LOW
3	2	1	0	Falling edge, SCLK idles HIGH
4	3	1	1	Rising edge, SCLK idles HIGH

Table 3: SPI Clock Modes Defined by CPOL and CPHA

This project's SPI decoder, like the SPI target block used for the FPGA-to-RP2040 host interface, is parameterized by CPOL and CPHA, allowing the same hardware shift logic to be reused for any of the four standard SPI modes.

3.3.4 SPI Frame Structure

Unlike UART (which delimits frames with start/stop bits) and I²C (which delimits frames with START/STOP bus conditions), an SPI frame is delimited entirely by the chip-select signal:

1. **CS Assertion:** The master drives CS LOW, selecting the target slave and marking the beginning of a transaction. No clock activity is required before this point.
2. **Clocked Bit Exchange:** The master toggles SCLK at the configured frequency. On each active sampling edge (determined by CPOL/CPHA), the current MOSI bit is sampled by the slave and the current MISO bit is sampled by the master, simultaneously. Data is conventionally transmitted MSB-first, though some devices use LSB-first ordering.
3. **Byte Boundaries:** SPI itself has no inherent byte framing; the transferred word width (commonly 8 bits, but sometimes 16 or 32) is a convention agreed upon by both devices, not a property enforced by the wire protocol. A decoder must therefore be told (or configured) how many bits constitute one transferred word.
4. **CS De-assertion:** The master drives CS HIGH, ending the transaction. Any partially shifted byte at the moment of de-assertion is generally considered invalid and discarded.

Because SPI has no acknowledgement mechanism analogous to I²C's ACK/NACK bit, the protocol provides no built-in error detection; reliability is the responsibility of the higher-level application protocol layered on top of the raw byte exchange.

3.3.5 SPI Decoder Design Implications

For the purposes of hardware decoding, SPI presents a different synchronization challenge than UART or I²C: the bit-sampling reference is the externally-generated SCLK edge, exactly as with I²C's SCL, but *two* data lines (MOSI and MISO) must be captured in parallel on every active edge, and the CS line must be continuously monitored for frame boundaries independently of SCLK activity. The FPGA SPI decoder implemented in this project (Section 7.4) addresses these requirements through dedicated two-stage synchronizers on CS, SCLK, MOSI, and MISO, CS-edge-triggered frame boundary detection, and a parallel dual shift-register structure that captures MOSI and MISO simultaneously on every synchronized SCLK sampling edge.

3.4 Digital Metastability and Synchronizer Design

3.4.1 The Metastability Problem in Digital Circuits

Every sequential digital element — flip-flop, latch, or register — is governed by two critical timing constraints relative to its active clock edge:

- **Setup time** (t_{su}): The minimum time before the active clock edge that the data input must be stable and not changing.
- **Hold time** (t_h): The minimum time after the active clock edge that the data input must remain stable and not changing.

Together, these constraints define a **setup-hold window** straddling the clock edge. If an input signal transitions within this window — which is possible whenever data originates from an unrelated clock domain — the flip-flop cannot unambiguously resolve its output to a clean logic HIGH or logic LOW. Instead, its output enters a **metastable state**: an intermediate analog voltage level (typically near the device's switching threshold, $V_{DD}/2$) that is not a valid logic level. In this state, the positive feedback mechanism of the cross-coupled inverter pair that forms the flip-flop's storage element has not yet amplified the ambiguous state to a clean rail voltage.

A metastable flip-flop output can remain at its intermediate voltage for an arbitrarily long time before eventually resolving to one of the two valid logic states. More critically, when a metastable output is consumed by downstream logic, different paths through the combinational logic may observe different resolved values from the same metastable signal, causing the entire system to enter an undefined state that may persist for many clock cycles.

3.4.2 Why Metastability is Unavoidable in This Project

In this system, the UART RX signal, I²C SDA, I²C SCL, and the SPI MOSI, MISO, SCLK, and CS signals all originate from the ESP8266, which has *no timing relationship whatsoever* to the ForgeFPGA system clock. The ESP8266 runs from its own internal oscillator at 80 MHz or 160 MHz, completely independently of the FPGA's internal clock. As a result, signal transitions on these lines can occur at any arbitrary phase relative to the FPGA clock edge. Statistically, at any clock edge, there is a non-zero probability that the signal is changing, placing the FPGA flip-flop exactly within its setup-hold window. This is not a design defect; it is a fundamental physical consequence of sampling an asynchronous signal.

3.4.3 Probabilistic Characterization: Mean Time Between Failures

The probability that a flip-flop remains in metastability for longer than a resolution time τ (i.e., has not resolved by time τ after the clock edge) is modelled exponentially:

$$P(\text{metastable for } > \tau) = T_0 \cdot f_D \cdot f_C \cdot e^{-\tau/\tau_c}$$

where:

- f_D = data toggle rate of the asynchronous input (Hz)
- f_C = clock frequency of the sampling flip-flop (Hz)
- T_0 = technology-dependent metastability constant (seconds)
- τ_c = metastability resolution time constant of the flip-flop (seconds)

The Mean Time Between Failures (MTBF) due to unresolved metastability is:

$$\text{MTBF} = \frac{e^{\tau/\tau_c}}{T_0 \cdot f_C \cdot f_D}$$

This equation reveals the key engineering insight: because τ appears in the *exponent*, every additional unit of resolution time (τ_c) provided to the flip-flop multiplies the MTBF by e (approximately 2.718). A two-stage synchronizer provides one full additional clock period of resolution time beyond the single-flip-flop case, which increases MTBF by a factor of e^{T_{clk}/τ_c} . For modern CMOS processes where τ_c is on the order of tens of picoseconds and T_{clk} at 10 MHz is 100 nanoseconds, this factor is astronomically large — making metastability failures astronomically unlikely well within two clock cycles.

3.4.4 Two-Stage Flip-Flop Synchronizer

The classical and widely adopted solution for single-bit asynchronous clock-domain crossing is the **two-stage (double-register) synchronizer**:

$$D_{async} \rightarrow \boxed{FF_1} \uparrow_{CLK} \xrightarrow{Q_1} \boxed{FF_2} \uparrow_{CLK} \xrightarrow{Q_{sync}} \text{Decoder Logic}$$

The mechanism is as follows:

- **First flip-flop (FF_1):** Captures the asynchronous input on each rising FPGA clock edge. Its output Q_1 may enter a metastable state whenever the input changes near the clock edge. However, Q_1 is not used by any combinational decoder logic; it feeds only FF_2 .
- **Second flip-flop (FF_2):** Samples Q_1 on the *next* rising FPGA clock edge, exactly one full clock period after FF_1 captured the input. During this full clock period, the metastable state of FF_1 's output has with overwhelming probability resolved to a clean logic value. The output Q_{sync} of FF_2 is therefore reliable and metastability-safe.

The synchronizer introduces a pipeline latency of exactly two clock cycles. At 10 MHz, this is 200 ns, which is negligible for UART signals (bit period $\approx 104 \mu\text{s}$), I²C signals (bit period $\approx 10 \mu\text{s}$ at 100 kHz), and SPI signals at the clock rates used in this project's validation experiments.

3.4.5 Synthesis Constraint for Synchronizer Placement

FPGA synthesis tools may, by default, optimize register placement without awareness of synchronizer function. If FF_1 and FF_2 are placed in physically distant logic slices, the routing delay between them could exceed the clock period, preventing FF_1 's metastable state from resolving before FF_2 samples it. The **Renesas Go Configure toolchain** automatically recognizes the two-stage synchronizer pattern and constrains FF_1 and FF_2 to the same physical logic slice, minimizing the routing delay between them and maximizing the resolution time available to FF_1 .

4 Hardware Platform

4.1 Renesas SLG47910 ForgeFPGA — Primary Protocol Analysis Engine

The Renesas SLG47910 ForgeFPGA serves as the primary computational and protocol-analysis engine of the Peripheral Analyzer Sniffer. Unlike conventional microcontroller-based protocol analyzers, where serial communication decoding is performed through software routines, this project implements the comprehensive protocol-monitoring pipeline directly in reconfigurable FPGA hardware. Consequently, all timing-critical operations are executed deterministically at the hardware level, independent of software execution latency, interrupt servicing, or processor workload.

Within the proposed architecture, the FPGA is responsible for every stage of protocol acquisition and processing. Incoming UART, I²C, and SPI signals generated by the ESP8266 are first captured through dedicated FPGA input pins and synchronized to the internal FPGA clock domain using two-stage flip-flop synchronizers. Following synchronization, protocol-specific hardware state machines perform UART frame reconstruction; I²C START and STOP detection, address extraction, and data extraction; and SPI chip-select edge detection, SCLK-synchronized bit shifting, and simultaneous MOSI/MISO frame reconstruction. Event generation and packet buffering follow uniformly for all three protocols. Decoded events are subsequently serialized and transferred to the RP2040 through the board's internal SPI communication path.

The major FPGA resources utilized in this project are summarized below:

- **Configurable Logic Blocks (CLBs):** The fundamental building blocks of the FPGA consist of configurable logic elements containing Look-Up Tables (LUTs), routing resources, and sequential storage elements. LUTs are used to implement combinational functions such as edge detection, protocol-state transitions, packet-generation logic, and decoder control paths, while the associated flip-flops implement state registers, counters, synchronizers, and buffering structures.
- **Sequential Logic Resources:** The FPGA provides edge-triggered storage elements used extensively throughout the design. These registers form the basis of the UART receiver state machine, I²C decoder state machine, SPI decoder state machine, FIFO control logic, packet serializer, bit counters, baud-rate timing generators, and synchronizer stages.
- **Internal Clocking Resources:** The device incorporates an internal oscillator that provides the global system clock used throughout the design. All decoder modules, counters, finite-state machines, and synchronization circuits operate from this common clock source, ensuring deterministic timing behaviour across the complete analysis pipeline.
- **Input Monitoring Infrastructure:** FPGA pins are utilized as protocol-monitoring inputs for seven distinct signal lines: UART TX, I²C SDA, I²C SCL, SPI MOSI, SPI MISO, and SPI SCLK/CS. Since these signals originate from an external ESP8266 operating in an unrelated clock domain, synchronization logic is required before they can be processed safely.
- **Clock-Domain Crossing Protection:** All monitored inputs — UART, I²C SDA/SCL, and SPI MOSI/MISO/SCLK/CS — pass through dedicated two-stage flip-flop synchronizers before entering protocol-decoding logic. This architecture provides robust metastability mitigation and ensures reliable operation of downstream state machines when sampling asynchronous external signals.
- **Hardware Protocol Decoders:** Dedicated RTL modules implement UART, I²C, and SPI decoding directly in hardware. The UART decoder performs start-bit detection, midpoint sampling, frame reconstruction, and byte extraction; the I²C decoder performs START detection, STOP detection, address extraction, ACK monitoring, and data-byte reconstruction; and the SPI decoder performs CS edge detection, SCLK synchronization, simultaneous MOSI/MISO bit shifting, and full-duplex byte reconstruction.
- **Synchronous FIFO Buffering:** Decoded protocol events from the UART, I²C, and SPI decoders are temporarily stored inside a common on-FPGA synchronous FIFO structure. This buffering mechanism decouples real-time protocol decoding from host-side packet retrieval, preventing data loss during bursts of communication activity while avoiding overflows.
- **SPI Target Interface:** The FPGA implements an SPI target interface that exports decoded protocol events to the RP2040. Structured event packets generated by the protocol-analysis engine are serialized and made available to the RP2040 through this interface.
- **Go Configure FPGA Design Suite:** Development of the FPGA design is performed using the Renesas Go Configure toolchain. The environment supports RTL design entry, synthesis, pin assignment, timing analysis, bitstream generation, device programming, and hardware debugging.

The use of FPGA-based protocol processing provides several important advantages, including deterministic timing behaviour, parallel hardware execution, low-latency event generation, reliable clock-domain crossing protection, and the ability to extend the architecture to additional protocols such as CAN or LIN through incremental RTL development.

4.2 RP2040 Microcontroller — SPI-to-USB Communication Bridge

The Raspberry Pi RP2040 functions exclusively as a communication and visualization bridge between the FPGA and the host computer. In contrast to many embedded protocol analyzers where protocol processing is performed within firmware, the RP2040 in this project performs no raw-signal protocol decoding or edge timing measurement for any of the monitored protocols. All protocol intelligence resides entirely within the FPGA fabric.

The RP2040 receives pre-decoded event packets generated by the FPGA and converts these packets into human-readable diagnostic output that can be displayed on a host computer through a standard serial terminal or graphical user interface (GUI).

The primary RP2040 resources utilized in this project are summarized below:

- **Dual-Core ARM Cortex-M0+ Processor:** The RP2040 incorporates two ARM Cortex-M0+ processor cores operating at frequencies up to 133 MHz. Although capable of significantly more complex computation, only a small fraction of the available processing capability is required for packet acquisition and terminal formatting.

- **Hardware SPI Controller:** Communication with the FPGA is performed through the RP2040's hardware SPI peripheral configured in host mode. The SPI controller retrieves event packets generated by the FPGA and transfers them into firmware-accessible memory without requiring software-based bit manipulation.
- **On-Chip SRAM:** The RP2040 provides 264 kB of SRAM, which is more than sufficient for packet buffers, serial output formatting, and bridge-firmware execution.
- **USB 1.1 Full-Speed Device Controller:** An integrated USB controller enables direct connection to a host computer. Using the `arduino-pico` framework, the RP2040 is configured as a USB CDC virtual serial device, allowing decoded protocol activity to be viewed using any standard serial-terminal application or forwarded to a GUI application over USB serial communication.
- **Packet Processing and Formatting:** Firmware running on the RP2040 receives structured event packets from the FPGA, extracts protocol-specific information such as event type, address fields, and payload bytes, and converts this information into readable terminal messages.
- **Host-System Interface:** The RP2040 serves as the final interface layer between the FPGA and the user. By presenting protocol activity through a standard USB serial connection, the system requires no specialized host software and remains compatible with common terminal applications such as PuTTY, Tera Term, and Thonny, as well as future GUI front-ends.

The FPGA–RP2040 partitioning adopted in this project provides a clear separation of responsibilities. The FPGA performs all real-time protocol acquisition and decoding functions for UART, I²C, and SPI alike, while the RP2040 manages packet acquisition, USB serial communication, and GUI communication. This architecture minimizes firmware complexity, preserves deterministic protocol-analysis performance, and provides a scalable foundation for future multi-protocol extensions.

4.3 ESP8266 NodeMCU — Test Traffic Generator

The ESP8266 NodeMCU is a low-cost microcontroller development board based on the Espressif ESP8266 system-on-chip, featuring a Tensilica Xtensa LX106 processor and integrated wireless communication capabilities. In the present work, the ESP8266 is utilized exclusively as an external test traffic generator for validating the functionality of the FPGA-based protocol analysis system.

The use of a programmable traffic generator provides a repeatable and controllable testing environment, allowing the performance and correctness of the Multi-Protocol Peripheral Analyzer Sniffer to be evaluated under known communication scenarios.

4.4 Complete Hardware Components Reference

Table 4: Complete Hardware Component List

S.No.	Component	Role in System
1	Vicharak Shrike Lite	Main development platform containing: • Renesas SLG47910 ForgeFPGA – Protocol analysis engine (UART, I²C, SPI decoding, synchronization, FIFO buffering, packet generation, serialization) • RP2040 Microcontroller – SPI-to-USB communication bridge, packet formatting, terminal/GUI interface
2	ESP8266 NodeMCU	External UART, I ² C, and SPI traffic generator used for protocol validation and testing
3	Jumper Wires	Signal routing between ESP8266 and Shrike Lite protocol-monitoring inputs
4	USB Type-C Cable	Power delivery, FPGA programming, and USB CDC communication for the Shrike Lite board
5	USB Micro Cable	Power delivery and programming interface for the NodeMCU test traffic generator

5 Software Tools and Development Environment

5.1 Go Configure FPGA Design Suite (Renesas)

The Go Configure suite is Renesas's official integrated development and programming toolchain for the ForgeFPGA family of devices. It provides a complete workflow from RTL entry to physical device configuration:

- **I/O Planner:** A graphical interface for assigning logical signal names to physical FPGA device pins, specifying pin directions (input or output), and configuring bridge output-enable control signals. The I/O Planner generates a pin constraint file that the synthesis back-end uses to correctly map logical ports to physical pads.
- **RTL Design Editor:** A text editor with Verilog syntax highlighting, used for entering the complete Verilog HDL description of all FPGA logic blocks, including the UART, I²C, and SPI decoders. Supports hierarchical module instantiation, enabling the clean, modular design structure used in this project.
- **Synthesis and Bitstream Generator:** Runs logic synthesis (converting Verilog RTL to a gate-level netlist), technology mapping (mapping the netlist to the SLG47910's specific LUT and flip-flop resources), place-and-route (assigning logic to physical fabric locations and routing interconnections), and bitstream generation (converting the placed-and-routed design to a device configuration file). The resulting bitstream is written to the SLG47910 via USB.

5.2 Arduino IDE with arduino-pico Core

The Arduino IDE, augmented with the community-maintained `arduino-pico` core by Earle Philhower, provides the development environment for the RP2040 SPI host reader firmware. The `arduino-pico` core exposes the RP2040's hardware peripherals through the familiar Arduino API:

- **USB CDC Serial via `Serial`:** Provides a virtual serial port over the USB connection. Data written to `Serial` appears immediately in the connected PC terminal or GUI application.
- **Hardware SPI via `SPI Library`:** Configures and manages the RP2040's hardware SPI0 peripheral. This enables reliable, synchronous reception of the FPGA packet stream at high clock rates without any software bit-banging or timing constraints.
- **C/C++ Compilation:** The Arduino IDE compiles firmware using the `arm-none-eabi-gcc` cross-compiler targeting the Cortex-M0+ instruction set, producing optimized binary images uploaded to the RP2040.

5.3 Thonny IDE with MicroPython (ESP8266)

The Thonny IDE is used to write and interactively execute MicroPython scripts on the ESP8266 test traffic generator. MicroPython's built-in `machine.UART`, `machine.I2C`, and `machine.SPI` classes allow the ESP8266 to generate protocol traffic with simple, readable Python code, making it straightforward to modify test parameters (baud rate, device address, SPI mode, payload data) during development.

5.4 Serial Terminal (PuTTY)

The PuTTY serial terminal application is used on the host PC, connected to the RP2040's USB CDC virtual serial port (at 115200 baud) to display sniffer logs, and to the ESP8266's serial port (at 9600 baud) for manual data entry. PuTTY is utilized for keyboard input to generate interactive I²C commands, verifying the sniffer's real-time capture dynamically.

5.5 GitHub for Version Control and Collaboration

All project artefacts — FPGA Verilog HDL source files, RP2040 SPI host reader firmware, ESP8266 MicroPython test scripts, and documentation source — are maintained in a public Git repository on GitHub. Version control provides a full revision history of every design change, enables tagged releases for stable milestones, and serves as a public engineering portfolio entry.

6 System Architecture

6.1 Architectural Overview and Signal Flow

The complete system is partitioned into three major functional layers. The FPGA occupies the central and most computationally significant layer; the other layers exist to generate traffic or consume the decoded outputs. The end-to-end signal flow is as follows:

Signals (UART / I²C / SPI) → FPGA Synchronizers → Protocol Decoders → Packet Generator → FIFO → SPI Streaming Interface → RP2040 → USB Serial → GUI

1. **Signal Source and MCU Control Layer (ESP8266):** The ESP8266 microcontroller acts as an interactive I²C controller and UART/SPI signal generator. Running an Arduino firmware, it polls the Serial terminal for user keystrokes, stores them in a buffer, and transmits them over the I²C bus to target address 0x48. It also generates standard UART traffic at 9600 baud and SPI traffic with configurable mode and word width. These physical signal lines are routed to the FPGA's input monitoring pins.
2. **FPGA Protocol Analysis Layer (SLG47910 ForgeFPGA):** This is the primary protocol analysis engine of the system. All protocol detection (UART, I²C, and SPI), event packet generation, event buffering in a parameterizable FIFO, and SPI target serialization occur here in hardware gates. The FPGA functions as an SPI target, making the decoded event bytes available to an external host controller.
3. **Host SPI, USB Serial, and GUI Layer:** A host controller reads the buffered event data from the FPGA's SPI target interface. The received bytes are parsed, formatted as human-readable protocol traces, transmitted over USB serial communication, and displayed on the user's serial console or GUI application in real time.

6.2 Hardware-Software Partitioning Rationale

The decision to implement all signal synchronization, protocol detection, and frame decoding exclusively on the ForgeFPGA — rather than on a microcontroller — is grounded in four fundamental engineering considerations:

1. **Timing Determinism:** Hardware state machines in FPGA fabric execute with absolute clock-cycle determinism. Every clock edge triggers the same state-transition logic in exactly the same number of gate delays, guaranteed. In contrast, microcontrollers running firmware are subject to interrupt service routine latencies, I-cache misses, pipeline stalls, and operating system scheduler jitter (even in bare-metal Arduino). These non-deterministic delays accumulate over a multi-bit frame and can cause missed signal edges, shifted bit windows, and corrupted byte values — a particular concern for SPI, where clock rates can be substantially higher than I²C.
2. **Clock-Domain Crossing Safety:** Correctly resolving asynchronous-to-synchronous clock-domain crossings requires physical control over flip-flop placement and proximity. FPGA synthesis tools, guided by appropriate constraints, can place FF_1 and FF_2 of each synchronizer in adjacent logic cells to minimize routing delay and maximize metastability resolution time. A microcontroller's register file has no equivalent concept; its GPIO input synchronizers (if present at all) are fixed in silicon with parameters specified by the microcontroller manufacturer.
3. **Zero MCU Overhead:** Running a communication stack on the microcontroller generates frequent, non-deterministic interrupt events. If protocol decoding were implemented in MCU firmware, these interrupts could preempt the bit-sampling loop at precisely the wrong moment, causing missed signal edges and corrupted data. The FPGA FIFO pipeline absorbs all burst protocol activity autonomously and delivers only complete, fully-decoded events to the host, which can process them at leisure without any real-time constraint.
4. **Scalability and Protocol Independence:** Additional protocol decoders (CAN, 1-Wire, LIN) can be added as independent RTL blocks instantiated in the FPGA top-level module. The host/MCU firmware requires no modification because the interface packet format is protocol-agnostic: event type code 1 could mean "UART byte", event type 6 could mean "SPI byte", and the software's `switch` case in `formatEvent()` gains a new case without any timing-critical changes.

6.3 SPI Target Interface and Pin Routing

The FPGA exposes its internal event buffer to the host PC or controller using a synchronous Serial Peripheral Interface (SPI) configured in target mode. Specific FPGA pins on the Shrike Lite platform are mapped to physical input/output lines:

- `spi_ss_n` (Input): Active-low target select.
- `spi_sck` (Input): SPI clock signal from the host.
- `spi_mosi` (Input): Master Output Target Input line.
- `spi_miso` (Output): Target Output Master Input line.

- `spi_miso_en` (Output): Output enable for the MISO buffer (asserted when select is active).

This target interface decouples the FPGA’s internal decoding logic from the SPI transaction clock domain. When the select line is active, the SPI host initiates transactions, causing the FPGA to shift event bytes from its FIFO onto the MISO line at the SCK rate. Note that this SPI *target* interface, used purely for the FPGA-to-RP2040 host link, is architecturally distinct from the SPI *decoder* block (Section 7.4) that monitors externally generated SPI traffic on the ESP8266 link for protocol analysis.

7 FPGA Design

7.1 Overview of the Complete FPGA Design

The SLG47910 ForgeFPGA implements seven major functional blocks that together constitute the complete, end-to-end multi-protocol analysis pipeline:

1. **UART Receiver Core Block** (`uart_rx_core.v`) — A four-state hardware FSM that performs clock synchronization, start-bit confirmation, bit-by-bit sampling, and byte framing.
2. **I²C Decoder Block** (`i2c_decoder.v`) — An edge-triggered hardware FSM that detects START/STOP conditions, shifts in SDA bits on SCL rising edges, and decodes addresses, data bytes, and ACK/NACK status.
3. **SPI Decoder Block** (`spi_decoder.v`) — A CS-edge-triggered, SCLK-synchronized hardware FSM that simultaneously captures MOSI and MISO bits and reconstructs full-duplex SPI byte frames.
4. **Event Packet Generator** (within `top.v`) — A write-sequencing state machine that translates UART and I²C decoder events into a common framed packet format for the FIFO.
5. **UART FIFO Buffer** (`uart_fifo.v`) — A synchronous circular FIFO with parameterizable depth and data width, buffering UART and I²C events.
6. **SPI Streaming Interface** (`spi_target.v`) — A fully synchronized SPI target interface that shifts serial data based on the host SPI clock (SCK), supporting variable widths, CPHA, CPOL, and LSB/MSB direction, used to stream the FIFO contents out to the RP2040.
7. **Top-Level Module** (`top.v`) — Integrates the UART receiver, I²C decoder, SPI decoder, FIFO buffer, and SPI streaming interface, orchestrating the event capture and transferring data from the FIFO to the SPI target.

All modules are clocked by the same FPGA system clock (`clk`) and share a common active-low synchronous reset (`rst_n`). No asynchronous combinational paths exist between the external inputs and any decoder state register.

7.2 Top-Level FPGA Module

7.2.1 Purpose and Architecture

The `top.v` module serves as the primary integration layer, instantiating all protocol decoder cores (UART, I²C, and SPI), the shared FIFO buffer, and the SPI target interface. It maps external physical pins to the internal logic and implements a central write-sequencing state machine (`tx_state`) that multiplexes asynchronous protocol events into a unified data stream for host consumption.

Resource Constraints: PWM Decoder Omission

Due to the logic element (LUT) and register capacity constraints inherent to the Renesas SLG47910 ForgeFPGA, the intended Pulse Width Modulation (PWM) decoder could not be fully implemented in this design iteration. While the top-level module retains the physical pin mapping and basic clock-domain synchronization logic for the `pwm_in` signal, the dedicated PWM decoder core was omitted to preserve essential routing resources for the full-duplex SPI capture pipeline.

7.2.2 Verilog Source Code

```

1 (* top *)
2 module top (
3     (* iopad_external_pin, clkbuf_inhibit *) input  clk,
4     (* iopad_external_pin *)                output clk_en,
5     (* iopad_external_pin *)                input  rst_n,
```



```

6
7      (* iopad_external_pin *)          input  uart_rx,
8      (* iopad_external_pin *)          input  i2c_sda,
9      (* iopad_external_pin *)          input  i2c_scl,
10     (* iopad_external_pin *)          input  pwm_in,
11     (* iopad_external_pin *)          input  spi_mon_cs_n,
12     (* iopad_external_pin *)          input  spi_mon_sclk,
13     (* iopad_external_pin *)          input  spi_mon_mosi,
14     (* iopad_external_pin *)          input  spi_mon_miso,
15
16     (* iopad_external_pin *)          input  spi_ss_n,
17     (* iopad_external_pin *)          input  spi_sck,
18     (* iopad_external_pin *)          input  spi_mosi,
19     (* iopad_external_pin *)          output spi_miso,
20     (* iopad_external_pin *)          output spi_miso_en,
21
22     (* iopad_external_pin *)          output reg led,
23     (* iopad_external_pin *)          output led_en
24 );
25
26 assign clk_en = 1'b1;
27 assign led_en = 1'b1;
28
29 reg pwm_meta, pwm_sync, pwm_d;
30 always @(posedge clk or negedge rst_n) begin
31     if(!rst_n) begin
32         pwm_meta <= 1'b0; pwm_sync <= 1'b0; pwm_d <= 1'b0;
33     end else begin
34         pwm_meta <= pwm_in; pwm_sync <= pwm_meta; pwm_d <= pwm_sync;
35     end
36 end
37 wire pwm_rise = pwm_sync & ~pwm_d;
38 wire pwm_fall = ~pwm_sync & pwm_d;
39
40 // UART
41 wire [7:0] rx_byte;
42 wire      rx_data_ready, framing_err;
43 uart_rx_core #(.CLKS_PER_BIT(5208)) u_uart (
44     .clk(clk), .rst_n(rst_n), .uart_rx(uart_rx),
45     .rx_byte(rx_byte), .rx_data_ready(rx_data_ready), .framing_err(framing_err)
46 );
47
48 // I2C
49 wire      pkt_valid, pkt_rw, pkt_ack, pkt_is_addr, pkt_start, pkt_stop;
50 wire [7:0] pkt_addr, pkt_data;
51 i2c_decoder u_i2c (
52     .clk(clk), .rst_n(rst_n),
53     .sda(i2c_sda), .scl(i2c_scl),
54     .pkt_valid(pkt_valid), .pkt_addr(pkt_addr), .pkt_data(pkt_data),
55     .pkt_rw(pkt_rw), .pkt_ack(pkt_ack), .pkt_is_addr(pkt_is_addr),
56     .pkt_start(pkt_start), .pkt_stop(pkt_stop)
57 );
58
59 // SPI
60 wire      spi_pkt_valid, spi_pkt_start, spi_pkt_stop;
61 wire [7:0] spi_pkt_mosi, spi_pkt_miso;
62 spi_decoder u_spi_decoder (
63     .clk(clk), .rst_n(rst_n),
64     .spi_mon_cs_n(spi_mon_cs_n), .spi_mon_sclk(spi_mon_sclk),
65     .spi_mon_mosi(spi_mon_mosi), .spi_mon_miso(spi_mon_miso),
66     .pkt_valid(spi_pkt_valid), .pkt_start(spi_pkt_start), .pkt_stop(
spi_pkt_stop),
67     .pkt_mosi(spi_pkt_mosi), .pkt_miso(spi_pkt_miso)
68 );
69
70 // Host
71 wire [7:0] host_rx_data;
72 wire      host_rx_valid, tx_data_hold;
73 reg       tx_hold_d;

```

```

74     wire            tx_hold_rise;
75
76     always @(posedge clk or negedge rst_n) begin
77         if(!rst_n) tx_hold_d <= 1'b0;
78         else      tx_hold_d <= tx_data_hold;
79     end
80     assign tx_hold_rise = tx_data_hold & ~tx_hold_d;
81
82     reg         fifo_wr_en;
83     reg [7:0]   fifo_wr_data;
84     reg [2:0]   tx_state;
85     reg [7:0]   saved_data;
86     reg [7:0]   saved_spi_miso;
87     reg         saved_ack;
88     reg         start_pending, stop_pending;
89     reg         spi_start_pending, spi_stop_pending, spi_valid_pending;
90     reg [7:0]   saved_spi_mosi;
91
92     localparam [2:0]
93         S_IDLE      = 3'd0,
94         S_UART_D    = 3'd1,
95         S_I2C_V     = 3'd2,
96         S_I2C_A     = 3'd3,
97         S_SPI_MOSI  = 3'd4,
98         S_SPI_MISO  = 3'd5;
99
100    always @(posedge clk or negedge rst_n) begin
101        if (!rst_n) begin
102            fifo_wr_en      <= 1'b0;
103            fifo_wr_data    <= 8'h00;
104            tx_state        <= S_IDLE;
105            saved_data      <= 8'h00;
106            saved_ack       <= 1'b0;
107            start_pending   <= 1'b0;
108            stop_pending    <= 1'b0;
109            spi_start_pending <= 1'b0;
110            spi_stop_pending <= 1'b0;
111            spi_valid_pending <= 1'b0;
112            saved_spi_mosi  <= 8'h00;
113            saved_spi_miso  <= 8'h00;
114        end else begin
115            fifo_wr_en      <= 1'b0;
116
117            if (pkt_start)    start_pending <= 1'b1;
118            if (pkt_stop)     stop_pending  <= 1'b1;
119            if (spi_pkt_start) spi_start_pending <= 1'b1;
120            if (spi_pkt_stop) spi_stop_pending  <= 1'b1;
121            if (spi_pkt_valid) begin
122                spi_valid_pending <= 1'b1;
123                saved_spi_mosi    <= spi_pkt_mosi;
124                saved_spi_miso    <= spi_pkt_miso;
125            end
126
127            if(tx_state == S_IDLE) begin
128                if(start_pending) begin
129                    start_pending <= 1'b0;
130                    if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <= 8'h01
; end
131                end
132                else if(pkt_valid) begin
133                    saved_ack <= pkt_ack;
134                    if(pkt_is_addr) begin
135                        saved_data <= pkt_addr;
136                        if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <=
8'h03; tx_state <= S_I2C_V; end
137                    end else begin
138                        saved_data <= pkt_data;
139                        if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <=
8'h04; tx_state <= S_I2C_V; end
140                    end

```

```

141         end
142         else if(stop_pending) begin
143             stop_pending <= 1'b0;
144             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <= 8'h02
; end
145         end
146         else if(rx_data_ready) begin
147             saved_data <= rx_byte;
148             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <= 8'hF1
; tx_state <= S_UART_D; end
149         end
150         else if(spi_start_pending) begin
151             spi_start_pending <= 1'b0;
152             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <= 8'hD0
; end
153         end
154         else if(spi_valid_pending) begin
155             spi_valid_pending <= 1'b0;
156             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <= 8'hD1
; tx_state <= S_SPI_MOSI; end
157         end
158         else if(spi_stop_pending) begin
159             spi_stop_pending <= 1'b0;
160             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <= 8'hD2
; end
161         end
162     end
163
164     case (tx_state)
165         S_UART_D: begin
166             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <=
saved_data; tx_state <= S_IDLE; end
167         end
168         S_I2C_V: begin
169             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <=
saved_data; tx_state <= S_I2C_A; end
170         end
171         S_I2C_A: begin
172             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <=
saved_ack ? 8'h05 : 8'h06; tx_state <= S_IDLE; end
173         end
174         S_SPI_MOSI: begin
175             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <=
saved_spi_mosi; tx_state <= S_SPI_MISO; end
176         end
177         S_SPI_MISO: begin
178             if(!fifo_full) begin fifo_wr_en <= 1'b1; fifo_wr_data <=
saved_spi_miso; tx_state <= S_IDLE; end
179         end
180         default: tx_state <= S_IDLE;
181     endcase
182 end
183 end
184
185 // FIFO
186 wire [7:0] fifo_data;
187 wire fifo_empty, fifo_full, fifo_almost_empty, fifo_almost_full, fifo_overflow
, fifo_underflow;
188 reg fifo_rd_en;
189
190 uart_fifo #( .DATA_WIDTH(8), .DEPTH(16)) u_fifo (
191     .clk(clk), .rst_n(rst_n), .wr_en(fifo_wr_en), .wr_data(fifo_wr_data),
192     .rd_en(fifo_rd_en), .rd_data(fifo_data), .empty(fifo_empty), .full(
fifo_full),
193     .almost_empty(fifo_almost_empty), .almost_full(fifo_almost_full),
194     .overflow(fifo_overflow), .underflow(fifo_underflow)
195 );
196
197 reg [7:0] tx_data;
198 reg
pending_read;

```



```

199
200     always @(posedge clk or negedge rst_n) begin
201         if(!rst_n) begin
202             tx_data      <= 8'h00;
203             fifo_rd_en   <= 1'b0;
204             pending_read <= 1'b0;
205         end else begin
206             fifo_rd_en <= 1'b0;
207
208             if(pending_read) begin
209                 fifo_rd_en   <= 1'b1;
210                 pending_read <= 1'b0;
211             end
212
213             if(tx_hold_rise) begin
214                 if(!fifo_empty) begin
215                     tx_data      <= fifo_data;
216                     pending_read <= 1'b1;
217                 end else begin
218                     tx_data <= 8'h00;
219                 end
220             end
221         end
222     end
223
224     always @(posedge clk or negedge rst_n) begin
225         if(!rst_n) led <= 1'b0;
226         else if(rx_data_ready || pkt_valid || spi_pkt_valid)
227             led <= ~led;
228     end
229
230     // SPI Target
231     spi_target #(.CPOL(1'b0), .CPHA(1'b0), .WIDTH(8), .LSB(1'b0)) u_spi (
232         .i_clk(clk), .i_rst_n(rst_n), .i_enable(1'b1),
233         .i_ss_n(spi_ss_n), .i_sck(spi_sck), .i_mosi(spi_mosi),
234         .o_miso(spi_miso), .o_miso_oe(spi_miso_en),
235         .o_rx_data(host_rx_data), .o_rx_data_valid(host_rx_valid),
236         .i_tx_data(tx_data), .o_tx_data_hold(tx_data_hold)
237     );
238
239 endmodule

```

Listing 1: Top-Level Module — top.v

7.2.3 Detailed Code Explanation

Hardware Core Instantiations and Clocking

The top module assigns continuous active-high values to `clk_en` and `led_en` to ensure physical logic stability. It acts as the structural parent, instantiating the three primary protocol engines: `uart_rx_core`, `i2c_decoder`, and `spi_decoder`. It routes events from all three decoders to the central write-sequencing state machine.

Event Formatting State Machine (tx_state)

Unlike the raw SPI pipeline, UART and I²C packets are multiplexed into a single shared 16-byte FIFO. Because protocol events often require multi-byte descriptive structures (for example, identifying if a byte is an I²C Address vs I²C Data, and whether it was ACK'd), the module utilizes a 4-state Mealy FSM:

- **S_IDLE:** Continuously polls hardware flags. If a global event occurs (like I²C START, STOP, or SPI Capture Done), it immediately writes single-byte markers (`0x01`, `0x02`, `0xC0`) into the FIFO. If a data payload arrives, it transitions states.
- **S_UART_D:** Formats a UART capture by first writing a `0xF1` marker byte, then in the next cycle, writing the raw `rx_byte` payload.
- **S_I2C_V:** Distinguishes between I²C Address (`0x03`) and I²C Data (`0x04`) markers, then writes the actual payload byte.

- **S_I2C_A:** Follows **S_I2C_V** to append the hardware-detected ACK (0x05) or NACK (0x06) status.

This FSM guarantees atomicity. Once a multi-byte sequence begins, overlapping events from other protocols are queued safely in their respective hardware registers, ensuring the FIFO data stream never interleaves out of order.

Host SPI Target Read Interface

The top module streams the decoded bytes from the shared FIFO buffer directly to the RP2040 host over the SPI target interface. The logic utilizes a strict synchronous handshaking mechanism (**pending_read**) tied to the **tx_hold_rise** edge from the SPI target block. This guarantees that the FIFO read pointer (**fifo_rd_en**) only advances after the host has successfully clocked out the current byte, preventing data duplication or loss across clock domains.

7.3 UART Receiver Core Block

7.3.1 Operational Overview

- Detects the START bit falling edge that begins every UART frame.
- Validates the START bit by confirming the line is still LOW at the midpoint of the start-bit period (filtering noise glitches).
- Samples each of the 8 data bits at its exact midpoint using a clock-cycle counter.
- Validates the STOP bit HIGH level.
- On successful frame completion, pulses **byte_ready** HIGH for exactly one clock cycle to signal the FIFO to accept the new byte.

7.3.2 Verilog Source Code

```

1 module uart_rx_decoder (
2     input wire      clk,
3     input wire      rst_n,
4     input wire      rx_sync,
5     output reg      byte_ready,
6     output reg [7:0] decoded_byte
7 );
8     parameter BIT_PERIOD = 1042;
9     parameter HALF_PERIOD = 521;
10
11     reg [10:0] clk_cnt;
12     reg [1:0] state;
13     reg [2:0] bit_idx;
14     reg [7:0] shift_reg;
15
16     localparam STATE_IDLE = 2'b00,
17                STATE_START = 2'b01,
18                STATE_DATA = 2'b10,
19                STATE_STOP = 2'b11;
20
21     always @(posedge clk or negedge rst_n) begin
22         if (!rst_n) begin
23             state <= STATE_IDLE;
24             clk_cnt <= 0;
25             bit_idx <= 0;
26             shift_reg <= 0;
27             byte_ready <= 0;
28             decoded_byte <= 0;
29         end else begin
30             byte_ready <= 0;
31
32             case (state)
33
34                 STATE_IDLE: begin
35                     clk_cnt <= 0;
36                     bit_idx <= 0;
37                     if (rx_sync == 1'b0)

```

```

38         state <= STATE_START;
39     end
40
41     STATE_START: begin
42         if (clk_cnt == HALF_PERIOD) begin
43             if (rx_sync == 1'b0) begin
44                 clk_cnt <= 0;
45                 state <= STATE_DATA;
46             end else begin
47                 state <= STATE_IDLE;
48             end
49         end else begin
50             clk_cnt <= clk_cnt + 1;
51         end
52     end
53
54     STATE_DATA: begin
55         if (clk_cnt == BIT_PERIOD - 1) begin
56             clk_cnt <= 0;
57             shift_reg[bit_idx] <= rx_sync;
58             if (bit_idx == 7)
59                 state <= STATE_STOP;
60             else
61                 bit_idx <= bit_idx + 1;
62             end else begin
63                 clk_cnt <= clk_cnt + 1;
64             end
65         end
66
67     STATE_STOP: begin
68         if (clk_cnt == BIT_PERIOD - 1) begin
69             state <= STATE_IDLE;
70             if (rx_sync == 1'b1) begin
71                 byte_ready <= 1'b1;
72                 decoded_byte <= shift_reg;
73             end
74             end else begin
75                 clk_cnt <= clk_cnt + 1;
76             end
77         end
78
79         default: state <= STATE_IDLE;
80
81     endcase
82 end
83 end
84
85 endmodule

```

Listing 2: UART Hardware Decoder FSM — `uart_rx_decoder.v`

7.3.3 Explanation of the source code

Port Declarations — Lines 1–7

clk (input wire): The FPGA global system clock, assumed to be 10 MHz for all timing calculations in this module. Every register update in this module occurs synchronously on the rising edge of this signal.

rst_n (input wire): Active-low synchronous reset. When this signal is driven LOW, all internal registers are cleared to their initial values on the next rising clock edge, returning the FSM to STATE_IDLE. The `_n` suffix is the conventional Verilog notation for active-low signals.

rx_sync (input wire): The synchronized, metastability-safe UART RX signal, output from the `two_stage_sync` instance dedicated to the UART input. This signal is guaranteed to be a clean, stable logic value whenever it is sampled by this module's flip-flops.

byte_ready (output reg): A single-cycle pulse output. This signal is asserted HIGH for exactly one clock cycle when the FSM has successfully decoded a complete, valid UART byte. The FIFO event buffer monitors this signal as its write-enable input; a one-cycle HIGH pulse causes the FIFO to accept and store exactly one

event.

decoded_byte (output reg [7:0]): An 8-bit output register that holds the reconstructed byte value when **byte_ready** is asserted. The value is valid for at least one clock cycle after **byte_ready** goes HIGH. In subsequent clock cycles, **decoded_byte** retains its value until the next successful decode.

Timing Parameters — Lines 8–9

parameter BIT_PERIOD = 1042: This parameter specifies the number of FPGA clock cycles that constitute exactly one UART bit period at 9600 baud with a 10 MHz FPGA system clock. The calculation is: $\lceil 10,000,000/9,600 \rceil = 1042$ cycles. Declaring this as a **parameter** rather than hardcoding the value 1042 directly into expressions makes the module easily reconfigurable: to support 115200 baud, one simply changes this parameter to $\lceil 10,000,000/115,200 \rceil = 86$.

parameter HALF_PERIOD = 521: Half of **BIT_PERIOD**, representing the clock count from start-bit falling edge to the midpoint of the start bit. The midpoint sampling philosophy (Section 3.1.2) requires sampling the start bit at its centre to both validate it and position subsequent samples correctly. This value is $\lceil 1042/2 \rceil = 521$ cycles.

Internal Register Declarations — Lines 11–14

reg [10:0] clk_cnt: An 11-bit up-counter used as the baud-rate timing reference. It counts FPGA clock cycles within each FSM state. 11 bits are required because **BIT_PERIOD** = 1042, which requires $\lceil \log_2(1042) \rceil = 11$ bits to represent without overflow ($2^{10} = 1024 < 1042 \leq 2048 = 2^{11}$). Using too few bits would cause the counter to wrap around before reaching **BIT_PERIOD**, causing premature state transitions and incorrect bit sampling.

reg [1:0] state: A 2-bit register encoding the current FSM state. Two bits support up to four distinct states, exactly the four required by the UART decode sequence (IDLE, START, DATA, STOP). Using **localparam** symbolic names (defined below) instead of raw 2-bit values in the FSM logic prevents accidental state number errors and makes the code self-documenting.

reg [2:0] bit_idx: A 3-bit counter tracking which data bit (0 through 7) the FSM is currently sampling in **STATE_DATA**. Three bits are required to count from 0 to 7 inclusive ($2^3 = 8$ distinct values). The UART protocol transmits 8 data bits per frame (for 8N1 configuration), so this counter runs from 0 to 7.

reg [7:0] shift_reg: An 8-bit accumulation register that stores sampled data bits as they are collected. Unlike a typical shift register (where bits shift in from one end and push previous bits aside), this implementation uses random-access assignment: each bit is written directly to its indexed position in the register (**shift_reg[bit_idx] <= rx_sync**). Since UART transmits LSB-first, bit 0 is sampled first and stored at **shift_reg[0]**, bit 1 at **shift_reg[1]**, and so on, automatically assembling the correct byte order without any post-processing reversal.

State Constant Definitions — Lines 16–19

localparam declarations: These four lines define symbolic names for the FSM state encoding:

- **STATE_IDLE = 2'b00:** The quiescent state. The FSM is here whenever the UART line is idle (HIGH) or between frames.
- **STATE_START = 2'b01:** Entered when the FSM detects a potential start bit (falling edge). The FSM counts to **HALF_PERIOD** in this state to validate the start bit.
- **STATE_DATA = 2'b10:** Entered after start-bit validation. The FSM counts one **BIT_PERIOD** per data bit and samples eight bits.
- **STATE_STOP = 2'b11:** Entered after the eighth data bit. The FSM counts one **BIT_PERIOD** to reach the stop-bit sampling window.

Using **localparam** (local parameter) rather than **parameter** prevents these values from being overridden by parent-module instantiation parameters; they are true compile-time constants visible only within this module.

Always Block Sensitivity List and Reset Logic — Lines 21–29

Line 21: always @(posedge clk or negedge rst_n)

This sensitivity list specifies two triggering conditions. **posedge clk** means the block executes on every rising clock edge for normal sequential operation. **negedge rst_n** means the block also executes when **rst_n** falls from HIGH to LOW, implementing an **asynchronous reset**: the reset takes effect immediately when **rst_n** is

asserted, without waiting for the next clock edge. Asynchronous reset is the standard choice for ensuring that an FPGA module reaches a known safe state as quickly as possible after power-up or system reset assertion.

Lines 22–28 (Reset branch): When `rst_n` is LOW (`!rst_n` is TRUE), all internal registers are simultaneously driven to their initial values:

- `state <= STATE_IDLE`: FSM returns to idle.
- `clk_cnt <= 0`: Timer cleared.
- `bit_idx <= 0`: Bit index reset to 0.
- `shift_reg <= 0`: Accumulator cleared.
- `byte_ready <= 0`: Output deasserted; prevents false FIFO writes.
- `decoded_byte <= 0`: Output cleared.

All assignments use non-blocking `<=` to model simultaneous parallel hardware register updates.

Line 30: `byte_ready <= 0`;

This is the **default deassert** in the normal (non-reset) clock branch. It guarantees that `byte_ready` is LOW in every clock cycle except the one specific cycle where it is explicitly set HIGH by the stop-bit validation logic. Without this default, the register would retain its HIGH value for multiple cycles after a successful decode, causing the FIFO to be written to multiple times for a single received byte.

STATE_IDLE Logic — Lines 32–38

In `STATE_IDLE`, the FSM is waiting for the UART line to go LOW, signalling the beginning of a start bit.

`clk_cnt <= 0; bit_idx <= 0;` These two lines reset the counter and bit index to zero on every clock cycle while in IDLE. This ensures that when the FSM leaves IDLE on start-bit detection, the counter starts from a known zero value. Without this explicit reset inside IDLE, a counter that was partially incremented during a glitch detection sequence (returning from `STATE_START` to IDLE) would produce an off-by-N error in the subsequent timing.

if (`rx_sync == 1'b0`): Checks whether the synchronized UART RX line has gone LOW. In Verilog, `1'b0` means a 1-bit binary value of zero. This condition is true on the first clock edge after `rx_sync` transitions from HIGH to LOW, i.e., on the first clock cycle of the potential start bit. Note that this is not a true edge-detection circuit (no comparison with a previous-cycle register); it simply observes the current logic level. Since the FSM only reaches this check while in IDLE (the line should be HIGH between frames), seeing a LOW is sufficient to indicate a falling edge.

`state <= STATE_START`: Non-blocking assignment that schedules the state transition. The FSM will be in `STATE_START` on the *next* clock edge.

STATE_START Logic — Lines 40–50

In `STATE_START`, the FSM is counting up to the midpoint of the start bit to validate that the line is genuinely a start bit and not a short noise glitch.

if (`clk_cnt == HALF_PERIOD`): When the counter reaches 521 (`HALF_PERIOD`), the FSM is at the midpoint of the start-bit interval. This is the sampling point.

Inner if (`rx_sync == 1'b0`) — valid start: If `rx_sync` is still LOW at the midpoint, the signal has been continuously LOW for at least half a bit period. This confirms that the initial LOW transition was a genuine start bit, not an electromagnetic noise spike shorter than `HALF_PERIOD` cycles. The counter is reset to 0 and the FSM transitions to `STATE_DATA`.

Inner else — noise glitch rejection: If `rx_sync` has returned HIGH before reaching the midpoint, the initial LOW was too short to be a valid start bit. The FSM returns directly to `STATE_IDLE` without decoding. This glitch rejection is critical in electrically noisy environments.

else `clk_cnt <= clk_cnt + 1`: While the counter has not yet reached `HALF_PERIOD`, it is incremented by 1 each clock cycle.

STATE_DATA Logic — Lines 52–63

In `STATE_DATA`, the FSM captures 8 consecutive data bits, one per `BIT_PERIOD` clock cycles.

if (`clk_cnt == BIT_PERIOD - 1`): The condition triggers when `clk_cnt` has counted 1041 cycles (since

it was reset to 0 on the last sample point). This places the sampling event at `BIT_PERIOD` clock cycles after the previous sample, i.e., at the exact centre of the next data bit. Using `BIT_PERIOD - 1` rather than `BIT_PERIOD` accounts for the counter reset occurring in the same clock cycle as the previous sample.

`clk_cnt <= 0`: Resets the counter immediately upon reaching the sample point, so it begins counting toward the next bit period sample instantly.

`shift_reg[bit_idx] <= rx_sync`: Captures the current synchronized RX line value directly into the bit position indexed by `bit_idx`. Since UART transmits LSB-first, `bit_idx` starts at 0 and increments to 7; the first bit sampled goes into `shift_reg[0]`, the second into `shift_reg[1]`, automatically assembling the byte in correct bit order.

`if (bit_idx == 7) state <= STATE_STOP`: After the 8th data bit (index 7) is captured, all data bits have been collected. The FSM transitions to `STATE_STOP` to verify and output the result.

`else bit_idx <= bit_idx + 1`: For the first 7 data bits, the index is incremented to point to the next bit position to be filled.

STATE_STOP Logic — Lines 65–75

In `STATE_STOP`, the FSM waits for the stop-bit sampling window and validates the stop bit before outputting the decoded byte.

`if (clk_cnt == BIT_PERIOD - 1)`: Counts one full `BIT_PERIOD` from the last data-bit sample to reach the midpoint of the stop-bit period.

`state <= STATE_IDLE`: The FSM always returns to `IDLE` from `STOP`, regardless of stop-bit validity.

`if (rx_sync == 1'b1) — valid stop bit`: The stop bit must be HIGH per the UART specification. If it is HIGH, the frame is complete and valid. `byte_ready` is pulsed HIGH for this one clock cycle, and `decoded_byte` is loaded with the contents of `shift_reg`. The FIFO's write-enable input, connected to `byte_ready`, detects this one-cycle pulse and stores the event.

Implicit else — framing error: If the stop bit is LOW, this is a framing error (Section 3.1.4). The decoded byte is silently discarded. `byte_ready` remains LOW (as set by the default deassert), so the FIFO is not written. The FSM returns to `IDLE` to resynchronize.

default Case — Line 77

`default: state <= STATE_IDLE`

This case handles the theoretically impossible situation where `state` contains a value not matching any of the four defined `localparam` encodings (e.g., due to an initial power-up condition or FPGA configuration transient). Forcing the state to `STATE_IDLE` ensures the FSM always converges to a valid operating state. This is a critical design safety practice for all hardware FSMs.

7.4 I²C Decoder Block

7.4.1 Operational Overview

The I²C decoder monitors both the synchronized SDA and SCL signals simultaneously. Its operation is fundamentally different from the UART decoder because I²C is synchronous: bit timing is determined by the SCL clock edges, not by counting internal clock cycles. The decoder's primary tasks are:

- Detecting START and STOP bus conditions by monitoring SDA transitions while SCL is HIGH.
- On each rising edge of SCL (while a transaction is in progress), capturing the value of SDA and shifting it into an 8-bit accumulator, MSB-first.
- On the 9th SCL rising edge, reading the ACK/NACK bit from SDA and emitting the appropriate decoded event (`EV_ADDR` or `EV_DATA`) with all associated metadata.

7.4.2 Verilog Source Code

```

1 module i2c_decoder (
2     input wire      clk,
3     input wire      rst_n,
4     input wire      sda_sync,
5     input wire      scl_sync,
6     output reg      event_valid,

```

```

7   output reg   [2:0] event_type,
8   output reg   [7:0] event_data,
9   output reg           event_ack,
10  output reg           event_rw
11 );
12  reg prev_sda, prev_scl;
13
14  always @(posedge clk or negedge rst_n) begin
15      if (!rst_n) begin
16          prev_sda <= 1'b1;
17          prev_scl <= 1'b1;
18      end else begin
19          prev_sda <= sda_sync;
20          prev_scl <= scl_sync;
21      end
22  end
23
24  wire sda_falling = (prev_sda == 1'b1) && (sda_sync == 1'b0);
25  wire sda_rising  = (prev_sda == 1'b0) && (sda_sync == 1'b1);
26  wire scl_rising  = (prev_scl == 1'b0) && (scl_sync == 1'b1);
27
28  wire start_detect = sda_falling && scl_sync;
29  wire stop_detect  = sda_rising  && scl_sync;
30
31  reg [3:0] bit_cnt;
32  reg [7:0] shift_reg;
33  reg      in_transaction;
34  reg      is_addr_phase;
35
36  localparam EV_START = 3'd1,
37             EV_STOP  = 3'd2,
38             EV_ADDR  = 3'd3,
39             EV_DATA  = 3'd4;
40
41  always @(posedge clk or negedge rst_n) begin
42      if (!rst_n) begin
43          in_transaction <= 1'b0;
44          is_addr_phase  <= 1'b0;
45          bit_cnt        <= 0;
46          shift_reg      <= 0;
47          event_valid    <= 1'b0;
48          event_type     <= 0;
49          event_data     <= 0;
50          event_ack      <= 1'b0;
51          event_rw       <= 1'b0;
52      end else begin
53          event_valid <= 1'b0;
54
55          if (start_detect) begin
56              in_transaction <= 1'b1;
57              is_addr_phase  <= 1'b1;
58              bit_cnt        <= 0;
59              event_valid    <= 1'b1;
60              event_type     <= EV_START;
61
62          end else if (stop_detect) begin
63              in_transaction <= 1'b0;
64              is_addr_phase  <= 1'b0;
65              event_valid    <= 1'b1;
66              event_type     <= EV_STOP;
67
68          end else if (in_transaction && scl_rising) begin
69              if (bit_cnt < 8) begin
70                  shift_reg <= {shift_reg[6:0], sda_sync};
71                  bit_cnt  <= bit_cnt + 1;
72              end else begin
73                  bit_cnt    <= 0;
74                  event_valid <= 1'b1;
75
76                  if (is_addr_phase) begin

```



```

77         event_type    <= EV_ADDR;
78         event_data    <= shift_reg[7:1];
79         event_rw      <= shift_reg[0];
80         event_ack     <= sda_sync;
81         is_addr_phase <= 1'b0;
82     end else begin
83         event_type <= EV_DATA;
84         event_data <= shift_reg;
85         event_ack  <= sda_sync;
86     end
87 end
88 end
89 end
90 end
91
92 endmodule

```

Listing 3: I²C Hardware Decoder FSM — `i2c_decoder.v`

7.4.3 Explanation of the source code

Port Declarations — Lines 1–11

`sda_sync` and `scl_sync` (input wire): The synchronized, metastability-safe versions of the I²C SDA and SCL signals respectively. Both are outputs from their dedicated `two_stage_sync` instances.

`event_valid` (output reg): A single-cycle pulse asserted HIGH whenever a complete, decoded event is ready to be written to the FIFO. This signal serves the same role as `byte_ready` in the UART decoder.

`event_type` (output reg [2:0]): A 3-bit event type code identifying which of the four I²C event types has been decoded: EV_START (1), EV_STOP (2), EV_ADDR (3), or EV_DATA (4).

`event_data` (output reg [7:0]): The decoded payload byte. For EV_ADDR events, this holds the 7-bit slave address (bits [7:1] of the address frame, right-justified to bits [6:0]). For EV_DATA events, this holds the full 8-bit data byte.

`event_ack` (output reg): Captures the ACK/NACK result from the 9th SCL clock pulse. 0 = ACK (SDA was LOW, slave acknowledged); 1 = NACK (SDA was HIGH, no acknowledgement).

`event_rw` (output reg): Captures the direction bit from the address frame. 0 = Write transaction (master will write to slave); 1 = Read transaction (master will read from slave). This field is only meaningful for EV_ADDR events; it retains its value from the address frame for all subsequent EV_DATA events within the same transaction.

Edge Detection Register Bank — Lines 12–21

`reg prev_sda, prev_scl`: These two single-bit registers each store the synchronized signal values from the *previous* clock cycle. They are the foundation of the module's edge detection mechanism: by comparing the current-cycle value to the previous-cycle value, the module can detect the exact clock cycle on which a signal transitions.

The `always` block (Lines 13–21): On every rising clock edge (or asynchronous reset), `prev_sda` is updated to the current value of `sda_sync`, and `prev_scl` to the current value of `scl_sync`. This creates a one-cycle delayed copy of each signal, enabling the edge detection combinational logic below.

Edge Detection Combinational Logic — Lines 23–28

wire `sda_falling`: A combinational pulse that is HIGH for exactly one clock cycle when `prev_sda` was 1 (last cycle HIGH) and `sda_sync` is now 0 (current cycle LOW). This precisely detects the clock cycle on which the SDA line transitions from HIGH to LOW — the first part of both the START condition detection.

wire `sda_rising`: Symmetrically detects SDA transitioning from LOW to HIGH — the precondition for STOP condition detection.

wire `scl_rising`: Detects SCL transitioning from LOW to HIGH. Each such rising edge is a valid bit-sampling event per the I²C specification (SDA is sampled by all receivers on SCL rising edges).

wire `start_detect = sda_falling && scl_sync`: Implements the I²C START condition definition directly from the protocol specification: SDA falls while SCL is HIGH. Because both conditions must be simultaneously

true — `sda_falling` is a one-cycle pulse and `scl_sync` is the current-cycle SCL level — this signal is HIGH for at most one clock cycle, producing a clean, single-cycle START detection pulse.

`wire stop_detect = sda_rising && scl_sync`: Implements the STOP condition: SDA rises while SCL is HIGH. Produces a single-cycle STOP detection pulse.

Transaction State Variables — Lines 30–37

`reg [3:0] bit_cnt`: A 4-bit counter tracking how many SCL rising edges (bits) have been received since the beginning of the current 9-bit frame (8 data/address bits plus 1 ACK/NACK bit). Values 0–7 correspond to data bits; value 8 triggers the ACK/NACK capture and event emission. 4 bits support counts 0–15.

`reg [7:0] shift_reg`: An 8-bit serial-in, parallel-out shift register. On each SCL rising edge, the current SDA value is shifted in from the LSB position while existing bits shift left by one position: `shift_reg <= {shift_reg[6:0], sda_sync}`. This implements MSB-first reception: the first received bit ends up at bit 7 after all 8 bits have been shifted in, then moves toward bit 0 as more bits arrive. After 8 SCL rising edges, `shift_reg[7:0]` holds the complete received byte in correct order.

`reg in_transaction`: A flag set by START condition detection and cleared by STOP condition detection. It gates the bit-capture logic: bit capture only occurs when this flag is HIGH, preventing noise or idle SCL activity outside of transactions from producing spurious decode events.

`reg is_addr_phase`: A flag that distinguishes the first 9-bit frame after START (which carries a 7-bit address + R/W bit + ACK) from all subsequent 9-bit frames (which carry data bytes + ACK). Set HIGH by START detection, cleared after the address frame has been emitted.

START Condition Handling — Lines 52–58

When `start_detect` is HIGH (SDA fell while SCL was HIGH):

`in_transaction <= 1'b1`: Opens the transaction gate, enabling subsequent bit captures on SCL edges.

`is_addr_phase <= 1'b1`: Marks the next 9-bit frame as the address frame.

`bit_cnt <= 0`: Resets the bit counter in preparation for the first address-frame SCL edge.

`event_valid <= 1'b1; event_type <= EV_START`: Emits an EV_START event to the FIFO in this same clock cycle. This allows the host terminal output to display “Transaction START” at the beginning of each I²C transaction.

Bit Capture and Frame Completion — Lines 65–84

The `else if (in_transaction && scl_rising)` branch runs on each SCL rising edge during an active transaction.

`if (bit_cnt < 8)` — **data bit capture**: For the first 8 SCL rising edges of a frame, the SDA value is shifted into `shift_reg` MSB-first, and `bit_cnt` is incremented.

`else` — **9th clock, ACK/NACK capture**: When `bit_cnt` reaches 8, the 9th SCL rising edge has arrived. The module:

1. Resets `bit_cnt` to 0 in preparation for the next frame.
2. Asserts `event_valid` to write the event to the FIFO.
3. For the address phase: extracts `shift_reg[7:1]` as the 7-bit address (in I²C, the address field occupies bits 7:1 of the first byte), extracts `shift_reg[0]` as the R/W direction bit, captures `sda_sync` as the ACK/NACK, and clears `is_addr_phase`.
4. For subsequent data frames: captures `shift_reg` as the full 8-bit data byte and `sda_sync` as the ACK/NACK.

7.5 SPI Decoder Block

7.5.1 Operational Overview

The SPI decoder block is responsible for passively monitoring the asynchronous SPI bus signals (CS, SCLK, MOSI, MISO) generated by the ESP8266, decoding the serial bits into structured bytes, and sending the decoded events directly to the shared FIFO buffer. The block consists of a single module: the SPI Decoder (extttspi_decoder.v).

7.5.2 SPI Decoder Module (spi_decoder.v)

The spi_decoder module acts as the physical layer monitor. It observes the raw, asynchronous SPI signals and translates them into structured byte-level events (start of frame, data bytes, end of frame).

```

1  module spi_decoder (
2      input  wire      clk,
3      input  wire      rst_n,
4
5      input  wire      spi_mon_cs_n,
6      input  wire      spi_mon_sclk,
7      input  wire      spi_mon_mosi,
8      input  wire      spi_mon_miso,
9
10     output reg        pkt_valid,
11     output reg        pkt_start,
12     output reg        pkt_stop,
13     output reg [7:0]  pkt_mosi,
14     output reg [7:0]  pkt_miso
15 );
16
17     reg [3:0] cs_filter;
18     reg [2:0] sclk_sync;
19     reg [1:0] mosi_sync;
20     reg [1:0] miso_sync;
21
22     reg cs_clean;
23     reg cs_clean_d;
24
25     always @(posedge clk or negedge rst_n) begin
26         if (!rst_n) begin
27             cs_filter  <= 4'b1111;
28             cs_clean   <= 1'b1;
29             cs_clean_d <= 1'b1;
30             sclk_sync  <= 3'b000;
31             mosi_sync  <= 2'b00;
32             miso_sync  <= 2'b00;
33         end else begin
34             cs_filter <= {cs_filter[2:0], spi_mon_cs_n};
35
36             if (cs_filter[3:1] == 3'b111) cs_clean <= 1'b1;
37             else if (cs_filter[3:1] == 3'b000) cs_clean <= 1'b0;
38
39             cs_clean_d <= cs_clean;
40
41             sclk_sync <= {sclk_sync[1:0], spi_mon_sclk};
42             mosi_sync <= {mosi_sync[0], spi_mon_mosi};
43             miso_sync <= {miso_sync[0], spi_mon_miso};
44         end
45     end
46
47     wire cs_fall   = (cs_clean_d == 1'b1 && cs_clean == 1'b0);
48     wire cs_rise   = (cs_clean_d == 1'b0 && cs_clean == 1'b1);
49     wire sclk_rise = (sclk_sync[2:1] == 2'b01);
50
51     // Decoder
52     reg      active;
53     reg [2:0] bit_cnt;
54     reg      byte_seen;
55     reg      first_byte;
56     reg [7:0] mosi_shift;
57     reg [7:0] miso_shift;

```

```

58
59     always @(posedge clk or negedge rst_n) begin
60         if (!rst_n) begin
61             active      <= 1'b0;
62             bit_cnt     <= 3'd0;
63             byte_seen   <= 1'b0;
64             first_byte  <= 1'b0;
65             mosi_shift  <= 8'd0;
66             miso_shift  <= 8'd0;
67
68             pkt_start   <= 1'b0;
69             pkt_stop    <= 1'b0;
70             pkt_valid   <= 1'b0;
71             pkt_mosi    <= 8'd0;
72             pkt_miso    <= 8'd0;
73         end else begin
74             pkt_start <= 1'b0;
75             pkt_stop  <= 1'b0;
76             pkt_valid <= 1'b0;
77
78             if (cs_fall) begin
79                 active <= 1'b1;
80                 bit_cnt <= 3'd0;
81                 byte_seen <= 1'b0;
82                 first_byte <= 1'b1;
83                 mosi_shift <= 8'd0;
84                 miso_shift <= 8'd0;
85             end
86             else if (cs_rise) begin
87                 active <= 1'b0;
88                 if (byte_seen) begin
89                     pkt_stop <= 1'b1;
90                 end
91             end
92             else if (active && sclk_rise) begin
93                 mosi_shift <= {mosi_shift[6:0], mosi_sync[1]};
94                 miso_shift <= {miso_shift[6:0], miso_sync[1]};
95
96                 if (bit_cnt == 3'd7) begin
97                     bit_cnt <= 3'd0;
98                     pkt_mosi <= {mosi_shift[6:0], mosi_sync[1]};
99                     pkt_miso <= {miso_shift[6:0], miso_sync[1]};
100                    pkt_valid <= 1'b1;
101                    byte_seen <= 1'b1;
102
103                    if (first_byte) begin
104                        pkt_start <= 1'b1;
105                        first_byte <= 1'b0;
106                    end
107                end else begin
108                    bit_cnt <= bit_cnt + 3'd1;
109                end
110            end
111        end
112    end
113endmodule

```

Listing 4: SPI Decoder Module — spi_decoder.v

7.5.3 Detailed Code Explanation

Synchronization and Glitch Filtering

The raw SPI signals are asynchronous to the FPGA's internal clock (`clk`). To prevent metastability, the module uses multi-stage flip-flop synchronizers (`sclk_sync`, `mosi_sync`, `miso_sync`). The Chip Select line (`spi_mon_cs_n`) uses a 3-bit majority-style filter (`cs_filter[3:1]`). The cleaned signal (`cs_clean`) only toggles when the input remains stable for three consecutive clock cycles, preventing electrical noise from triggering false transactions.

Edge Detection

The module uses delayed signals to detect critical bus events: `cs_fall` detects when Chip Select goes LOW (starting a transaction), `cs_rise` detects when Chip Select goes HIGH (ending a transaction), and `sclk_rise` detects the rising edge of the SPI clock (sampling edge).

Byte Reconstruction FSM

When `cs_fall` occurs, internal counters and shift registers are initialized. On every `sclk_rise`, the current MOSI and MISO bits are shifted into their respective 8-bit shift registers. Once 8 bits have been received (`bit_cnt == 7`), the module asserts `pkt_valid` for one clock cycle and outputs the full bytes (`pkt_mosi` and `pkt_miso`). If this is the first byte of the transaction, `pkt_start` is also asserted. Upon `cs_rise`, `pkt_stop` is asserted to indicate the end of the transaction.

7.6 UART FIFO Buffer

7.6.1 Purpose and Architecture

The FIFO (First In, First Out) buffer acts as a decoupling layer between the real-time UART/I²C decoders and the slower SPI streaming interface. The UART and I²C FIFO buffer (`uart_fifo.v`) is a synchronous circular queue featuring parameterizable data width and depth, status indicators for queue occupancy (`empty`, `full`, `almost_empty`, `almost_full`), and single-cycle error strobes for buffer overrun (`overflow`) or underrun (`underflow`).

7.6.2 Verilog Source Code

```

1 module uart_fifo #(
2     parameter DATA_WIDTH      = 8,
3     parameter DEPTH            = 16,
4     parameter PTR_WIDTH        = $clog2(DEPTH),
5     parameter ALMOST_EMPTY_THRESH = 2,
6     parameter ALMOST_FULL_THRESH  = DEPTH - 4
7 ) (
8     input  wire      clk,
9     input  wire      rst_n,
10
11
12     input  wire      wr_en,
13     input  wire [DATA_WIDTH-1:0] wr_data,
14
15     input  wire      rd_en,
16     output wire [DATA_WIDTH-1:0] rd_data,
17
18     // Status flags
19     output wire      empty,
20     output wire      full,
21     output wire      almost_empty,
22     output wire      almost_full,
23
24     // Error strobes (single-cycle)
25     output reg       overflow,
26     output reg       underflow
27 );
28
29
30     reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];
31
32     reg [PTR_WIDTH-1:0] wr_ptr;
33     reg [PTR_WIDTH-1:0] rd_ptr;
34     reg [PTR_WIDTH:0]   count;
35
36     assign empty        = (count == {(PTR_WIDTH+1){1'b0}});
37     assign full         = (count == DEPTH[PTR_WIDTH:0]);
38     assign almost_empty = (count <= ALMOST_EMPTY_THRESH[PTR_WIDTH:0]);
39     assign almost_full  = (count >= ALMOST_FULL_THRESH[PTR_WIDTH:0]);
40
41
42     assign rd_data = mem[rd_ptr];

```

```

43
44
45     integer i;
46
47     always @(posedge clk or negedge rst_n) begin
48         if (!rst_n) begin
49             wr_ptr    <= {PTR_WIDTH{1'b0}};
50             rd_ptr    <= {PTR_WIDTH{1'b0}};
51             count     <= {(PTR_WIDTH+1){1'b0}};
52             overflow  <= 1'b0;
53             underflow <= 1'b0;
54             for (i = 0; i < DEPTH; i = i + 1)
55                 mem[i] <= {DATA_WIDTH{1'b0}};
56         end else begin
57
58             overflow  <= 1'b0;
59             underflow <= 1'b0;
60
61             case ({wr_en & ~full, rd_en & ~empty})
62
63                 2'b10: begin
64                     mem[wr_ptr] <= wr_data;
65                     wr_ptr      <= wr_ptr + 1'b1;
66                     count       <= count + 1'b1;
67                 end
68
69                 2'b01: begin
70                     rd_ptr <= rd_ptr + 1'b1;
71                     count <= count - 1'b1;
72                 end
73
74                 2'b11: begin
75                     mem[wr_ptr] <= wr_data;
76                     wr_ptr      <= wr_ptr + 1'b1;
77                     rd_ptr      <= rd_ptr + 1'b1;
78                 end
79
80                 end
81
82                 default: begin
83
84                     if (wr_en && full)
85                         overflow <= 1'b1;
86                     if (rd_en && empty)
87                         underflow <= 1'b1;
88                 end
89
90             endcase
91         end
92     end
93 endmodule

```

Listing 5: Synchronous Parameterized Queue — `uart_fifo.v`

7.6.3 Detailed Code Explanation

Parameters and Ports — Lines 1–28

DATA_WIDTH and **DEPTH**: Configure the width of the stored elements (defaults to 8 bits) and the queue capacity (defaults to 16 entries).

PTR_WIDTH: Calculated using the system function `$clog2(DEPTH)`, which returns the ceiling base-2 logarithm of the depth, matching pointer sizes (4 bits for 16 entries).

ALMOST_EMPTY_THRESH and **ALMOST_FULL_THRESH**: Define limits for early-warning threshold status flags.

wr_en (input) and **wr_data** (input): Write control lines. When **wr_en** is asserted, **wr_data** is placed into the queue.

rd_en (input) and rd_data (output): Read control lines. When **rd_en** is asserted, the queue advances to the next item. The output is combinationaly driven.

Flag Declarations and Continuous Assignments — Lines 31–40

empty and full: Continuous assignments check whether **count** equals 0 or **DEPTH** respectively. Note that the bit width of the comparison is explicitly padded to **PTR_WIDTH+1** to avoid syntax warnings.

almost_empty and almost_full: Indicate when the buffer is nearing empty or full states, allowing the host or writer to throttle transactions to prevent overruns.

rd_data = mem[rd_ptr]: Continuous assignment driving the read data bus directly from the location addressed by the read pointer, implementing a first-word fall-through (FWFT) read mode.

Reset and Initialization — Lines 46–55

Upon active-low asynchronous reset (**rst_n == 1'b0**):

- Pointer registers (**wr_ptr**, **rd_ptr**) and counter register (**count**) are cleared to 0.
- Error strobes (**overflow**, **underflow**) are forced to 0.
- An **integer** loop clears all storage locations in the **mem** array, preventing uninitialized memory warnings during simulation.

Read/Write FSM Case Block — Lines 60–88

The queue prioritizes write and read actions by matching control states in a 2-bit case selector, concatenating the active write request (**wr_en & ~full**) and read request (**rd_en & ~empty**):

- **2'b10** (Write Only): Places data at **wr_ptr**, increments **wr_ptr**, and increments **count**.
- **2'b01** (Read Only): Increments **rd_ptr** and decrements **count**.
- **2'b11** (Simultaneous Write/Read): Writes data to **wr_ptr**, increments both pointers, and leaves **count** unchanged.

Error Strobe Generation — Lines 90–94

If neither request is valid, the default case executes:

- If **wr_en** is asserted while the queue is full, the single-cycle **overflow** register is pulsed.
- If **rd_en** is asserted while the queue is empty, the single-cycle **underflow** register is pulsed.

Both error indicators automatically clear in the next clock cycle since they are explicitly assigned to 0 at the start of the clocked block (lines 57–58).

7.7 SPI Streaming Interface Block

7.7.1 Operational Overview

The SPI streaming interface block (**spi_target.v**) is a synchronous serial communication peripheral that permits an external host controller to read out the FIFO-buffered UART, I²C, and SPI events. It is parameterized by CPOL, CPHA, data width (defaults to 8 bits), and bit ordering (MSB/LSB first). Key architectural functions:

- **Input Synchronization:** Double-register synchronization is applied to the input SPI clock (**i_sck**) and target select (**i_ss_n**) signals to isolate clock domains.
- **w_sck_edge and w_sck_edge_op** detection: rising/falling edges are computed dynamically based on the CPOL and CPHA parameters.
- **Bit Counting:** Tracks data transmission bit-by-bit using a counter.
- **Shift Registers:** Performs serial-to-parallel conversion on the MOSI input (**o_rx_data**) and parallel-to-serial conversion for the MISO output (**r_miso_data**).
- **Hold/Request Signal:** Generates a single-cycle hold strobe (**o_tx_data_hold**) at the start of each byte transfer to request the next data byte from the FIFO.

7.7.2 Verilog Source Code

```

1 module spi_target #(
2     parameter CPOL    = 1'b0,    // When one, clock is low in idle, otherwise clock is
        high
3     parameter CPHA    = 1'b0,    // When one, sampling occurs at falling edge,
        otherwise at rising edge of non-inverted clock
4     parameter WIDTH   = 8,        // Determines the data width of SPI to the input and
        output data buses
5     parameter LSB     = 1'b0     // When one, data starts from LSB, otherwise data
        starts from MSB
6 ) (
7     // common ports
8     input            i_clk,        // input clock signal
9     input            i_rst_n,     // input negative reset signal
10    // control signal
11    input            i_enable,     // input enable SPI target signal
12    // SPI interface ports
13    input            i_ss_n,       // input target select signal
14    input            i_sck,        // input spi clock signal
15    input            i_mosi,       // input controller output target input
        signal
16    output           o_miso,        // output controller input target output
        signal
17    output           o_miso_oe,     // output miso enable output signal
18    //RX internal ports
19    output reg [WIDTH-1:0] o_rx_data, // output data bus
20    output reg         o_rx_data_valid, // output receive data valid signal
21    //TX internal ports
22    input [WIDTH-1:0] i_tx_data,    // input data bus
23    output           o_tx_data_hold // output signal used to get tx data
        from i_tx_data input
24 );
25
26 // Signal declaration
27 reg [2:0] r_ss_n_sync, r_sck_sync;
28 reg [$clog2(WIDTH-1):0] r_transmission_count;
29 reg [WIDTH-1:0] r_miso_data;
30 wire           w_sck_r_edge, w_sck_f_edge, w_sck_edge, w_sck_edge_op;
31
32 // SPI input signals synchronization
33 always @(posedge i_clk or negedge i_rst_n) begin
34     if (!i_rst_n) begin
35         r_ss_n_sync <= 'b111;
36     end else if (i_enable) begin
37         r_ss_n_sync <= {r_ss_n_sync[1:0], i_ss_n};
38     end else begin
39         r_ss_n_sync <= 'b111;
40     end
41 end
42
43 always @(posedge i_clk or negedge i_rst_n) begin
44     if (!i_rst_n) begin
45         r_sck_sync <= 'h0;
46     end else if (i_enable) begin
47         r_sck_sync <= {r_sck_sync[1:0], i_sck};
48     end else begin
49         r_sck_sync <= 'h0;
50     end
51 end
52
53 // Create rising and falling edge signals from spi_clk signal
54 assign w_sck_r_edge = ~r_sck_sync[2] & r_sck_sync[1];
55 assign w_sck_f_edge = r_sck_sync[2] & ~r_sck_sync[1];
56 assign w_sck_edge   = (CPHA^CPOL) ? w_sck_f_edge : w_sck_r_edge;
57 assign w_sck_edge_op = (CPHA^CPOL) ? w_sck_r_edge : w_sck_f_edge;
58
59 // Create transmission bit counter
60 always @(posedge i_clk or negedge i_rst_n) begin
61     if (!i_rst_n) begin

```

```

62     r_transmission_count <= 'h0;
63 end else if (!i_enable || r_ss_n_sync[1]) begin
64     r_transmission_count <= 'h0;
65 end else if (w_sck_edge) begin
66     if (r_transmission_count == WIDTH-1) begin
67         r_transmission_count <= 'h0;
68     end else begin
69         r_transmission_count <= r_transmission_count + 1;
70     end
71 end
72 end
73
74 // Create o_rx_data bus and valid signals
75 always @(posedge i_clk or negedge i_rst_n) begin
76     if (!i_rst_n) begin
77         o_rx_data <= 'h0;
78     end else if (w_sck_edge) begin
79         if (LSB) begin
80             o_rx_data <= {i_mosi, o_rx_data[WIDTH-1:1]};
81         end else begin
82             o_rx_data <= {o_rx_data[WIDTH-2:0], i_mosi};
83         end
84     end
85 end
86
87 always @(posedge i_clk or negedge i_rst_n) begin
88     if (!i_rst_n) begin
89         o_rx_data_valid <= 1'b0;
90     end else if (r_ss_n_sync[1] || (r_transmission_count == 0 && w_sck_edge)) begin
91         o_rx_data_valid <= 1'b0;
92     end else if (w_sck_edge && r_transmission_count == WIDTH-1) begin
93         o_rx_data_valid <= 1'b1;
94     end
95 end
96
97 // Create o_tx_data_hold signal
98 assign o_tx_data_hold =
99     (r_transmission_count == 0 & w_sck_edge_op);
100
101 // Create o_miso and OE signals
102 always @(posedge i_clk or negedge i_rst_n) begin
103     if (!i_rst_n) begin
104         r_miso_data <= 'h0;
105     end else if (o_tx_data_hold) begin
106         r_miso_data <= i_tx_data;
107     end else if (w_sck_edge_op) begin
108         if (LSB) begin
109             r_miso_data <= r_miso_data >> 1;
110         end else begin
111             r_miso_data <= r_miso_data << 1;
112         end
113     end
114 end
115
116 assign o_miso      = (LSB) ? r_miso_data[0] : r_miso_data[WIDTH-1];
117 assign o_miso_oe   = ~r_ss_n_sync[2];
118
119 endmodule

```

Listing 6: SPI Target Controller — spi_target.v

7.7.3 Detailed Code Explanation

Parameters and Ports — Lines 1–28

CPOL and CPHA: Define SPI clock polarity and phase respectively. Under the default configuration (00), SCK is low in idle, and data is captured on the rising edge.

WIDTH: Configures the data frame size (default is 8 bits).

i_ss_n, i_sck, i_mosi, and **o_miso:** Physical SPI bus interface connections.

i_tx_data (input [WIDTH-1:0]): The parallel byte from the FIFO to be transmitted over SPI.

o_tx_data_hold (output wire): Hold strobe indicating when the SPI block requires the next byte.

Clock and Select Synchronization — Lines 31–51

r_ss_n_sync and **r_sck_sync** (3-bit registers): Implement shift-register synchronization buffers for the async inputs. This helps isolate the internal system clock domain from the SPI clock domain and prevents metastable states from entering the internal logic.

Clock Edge Generation and Bit Counter — Lines 53–76

w_sck_edge and **w_sck_edge_op:** Continuous assignments compute the active data-capture edge (**w_sck_edge**) and shifting edge (**w_sck_edge_op**) dynamically based on CPHA and CPOL configuration.

r_transmission_count (register): Tracks the currently shifted bit index from 0 to **WIDTH-1**, resetting automatically when the frame completes.

Receive Register and Valid Signal — Lines 78–103

o_rx_data: Slightly shifts the bits on MOSI (**i_mosi**) either left or right (based on the **LSB** parameter) on each active SCK edge.

o_rx_data_valid: Pulsed high for one cycle when the last bit of a frame is received (**r_transmission_count** == **WIDTH-1**), signifying that **o_rx_data** is stable.

Transmit Strobe and Shift Register — Lines 105–127

o_tx_data_hold: Asserted when **r_transmission_count** == 0 at the shift clock edge, indicating that the shift register has just started transmitting a byte and requires the next byte to be fetched.

r_miso_data: Parallel-to-serial register. When **o_tx_data_hold** is asserted, it loads the data from **i_tx_data**. On subsequent shift clock edges (**w_sck_edge_op**), it shifts the contents left or right.

o_miso and **o_miso_oe:** **o_miso** exposes the current LSB or MSB of the shift register. **o_miso_oe** acts as the tri-state buffer enable driven by the select synchronizer (**~r_ss_n_sync[2]**), ensuring MISO goes high-impedance when select is inactive.

8 Firmware Architecture (Arduino MCU Code)

8.1 Microcontroller Role and Boundaries

The MCU (such as the ESP8266 NodeMCU) runs a minimal interactive firmware. Instead of implementing complex decoding logic, its primary function is to serve as a multi-protocol signal generator for UART, I²C, and SPI validation. By routing character inputs from a computer's serial terminal (like PuTTY) onto the respective bus, the firmware generates deterministic, manual transactions that are captured and verified in real time by the FPGA's protocol sniffer.

The design boundaries for this firmware are:

- The MCU **does not** monitor or parse the sniffer's SPI target output.
- The MCU **does not** implement protocol decoding or event marker parsing.
- The MCU **only** acts as a signal generator/transmitter, driving the UART TX, I²C SDA/SCL, or SPI MOSI/SCLK/CS lines to create test traffic.

8.2 Firmware Implementation

To validate all three protocols, three independent Arduino sketches were created. Depending on the test phase, the ESP8266 is flashed with the respective firmware.

8.2.1 UART Signal Generator

```
1 String lineBuffer = "";
2
3 void setup()
4 {
5     Serial.begin(9600);
6 }
7
8 void loop()
9 {
10     while (Serial.available())
11     {
12         char c = Serial.read();
13
14         if (c == '\r' || c == '\n')
15         {
16             if (lineBuffer.length() > 0)
17             {
18                 Serial.println(lineBuffer);
19                 lineBuffer = "";
20             }
21         }
22         else
23         {
24             lineBuffer += c;
25         }
26     }
27 }
```

Listing 7: Arduino UART Signal Generator — `uart_traffic_gen.ino`

8.2.2 I²C Signal Generator

```
1 #include <Wire.h>
2
3 #define I2C_ADDR 0x48
4
5 void setup()
6 {
7     Serial.begin(9600);
8     Wire.begin(D2, D1);
9 }
10
11 void loop()
12 {
13     while (Serial.available())
14     {
15         char c = Serial.read();
16
17         if (c == '\r' || c == '\n')
18         {
19             Serial.println();
20
21             continue;
22         }
23         Wire.beginTransmission(I2C_ADDR);
24         Wire.write((uint8_t)c);
25         Wire.endTransmission();
26     }
27 }
```

Listing 8: Arduino I²C Signal Generator — `i2c_traffic_gen.ino`

8.2.3 SPI Signal Generator

```

1  #include <SPI.h>
2
3  #define CS_PIN D8
4
5  String messageBuffer = "";
6
7  void setup()
8  {
9      Serial.begin(9600);
10
11     pinMode(CS_PIN, OUTPUT);
12     digitalWrite(CS_PIN, HIGH);
13
14     SPI.begin();
15 }
16
17 void loop()
18 {
19     while (Serial.available())
20     {
21         char c = Serial.read();
22
23         if (c == '\r' || c == '\n')
24         {
25             if (messageBuffer.length() > 0)
26             {
27                 digitalWrite(CS_PIN, LOW);
28
29                 for (int i = 0; i < messageBuffer.length(); i++)
30                 {
31                     SPI.transfer((uint8_t)messageBuffer[i]);
32                 }
33
34                 digitalWrite(CS_PIN, HIGH);
35
36                 messageBuffer = "";
37             }
38         }
39         else
40         {
41             messageBuffer += c;
42         }
43     }
44 }

```

Listing 9: Arduino SPI Signal Generator — spi_traffic_gen.ino

8.3 Detailed Logic Explanation

8.3.1 UART Signal Generator Explanation

String Accumulation and Transmission

The UART firmware leverages the built-in hardware serial peripheral. It polls `Serial.available()` to read incoming characters from the PC terminal. Instead of transmitting each character immediately, it appends them to a `lineBuffer` string. When a carriage return (`\r`) or newline (`\n`) is detected, the script blasts the entire accumulated string back over the UART TX line using `Serial.println()`. This allows the FPGA sniffer to capture a continuous, high-speed burst of back-to-back UART frames rather than isolated, slow keystrokes.

8.3.2 I²C Signal Generator Explanation

Real-Time Byte Transmission

Unlike the UART script, the I²C firmware generates transactions in real-time as the user types. It imports `<Wire.h>` and initializes the I²C bus on pins D2 (SDA) and D1 (SCL). For every valid character received from the serial terminal (ignoring newlines), it immediately constructs an I²C packet targeting a simulated slave address (`0x48`). It generates an I²C START condition via `Wire.beginTransmission()`, writes the single character payload onto the bus, and issues a STOP condition via `Wire.endTransmission()`.

8.3.3 SPI Signal Generator Explanation

Manual Chip Select and Burst Transmission

The SPI firmware imports `<SPI.h>` and manually configures pin D8 as an output to serve as the active-low Chip Select (CS) line. Similar to the UART script, it accumulates typed characters into a `messageBuffer` to enable high-speed burst transmissions. When a newline is pressed, the script manually pulls the CS pin LOW (starting the SPI transaction). It then iterates through the buffer, shifting out each character sequentially using `SPI.transfer()`, and finally pulls the CS pin HIGH to terminate the transaction. This mimics a true multi-byte SPI peripheral write operation for the FPGA to decode.

9 RP2040 MicroPython Host Reader

9.1 Role of the RP2040 in the Final System

The RP2040 microcontroller acts as the central intelligence and data presentation layer for the protocol analyzer. While the FPGA handles the high-speed, hard-real-time capture and buffering of bits, it lacks a natural way to display complex formatted strings to a user. The RP2040 bridges this gap by continuously polling the FPGA over a high-speed SPI bus, fetching the raw event packets, parsing the hexadecimal markers, and printing human-readable transaction summaries (like "START", "ADDR 0x24 (WRITE) ACK", or "MOSI: 0xAB MISO: 0xCD") directly to the computer's terminal.

In addition to parsing FPGA data, the RP2040 also runs a concurrent I²C slave state machine to actively participate in bus transactions for validation purposes.

9.2 Complete MicroPython Source Code

```

1  import time
2  from machine import Pin, SPI
3  import Shrike
4
5  # Vicharak Shrike Lite Pin Map
6  CS_PIN = 17
7  SCK_PIN = 18
8  MOSI_PIN = 19
9  MISO_PIN = 20
10
11 # GPIO Configuration
12 cs = Pin(CS_PIN, Pin.OUT)
13 cs.value(1)
14
15 # SPI Interface to FPGA (1 MHz, Mode 0)
16 spi = SPI(0, baudrate=1000000, polarity=0, phase=0,
17          sck=Pin(SCK_PIN), mosi=Pin(MOSI_PIN), miso=Pin(MISO_PIN))
18
19 # FPGA Programming Configuration
20 shrike = Shrike.ShrikeFlash()
21
22 # Configure RP2040 I2C Slave Validation Path
23 from machine import I2C
24 slave = I2C(1,
25            sck=Pin(15),
26            sda=Pin(14),
27            freq=100000,
28            slave_id=0x48
29 )

```

```

30
31 uart_message = ""
32 slave_message = ""
33 i2c_lines = []
34 spi_mosi_lines = []
35 spi_miso_lines = []
36
37 def spi_read(n):
38     tx = bytes([0] * n)
39     rx = bytearray(n)
40     cs.value(0)
41     spi.write_readinto(tx, rx)
42     cs.value(1)
43     return rx
44
45 print()
46 print("Protocol Analyzer")
47 print()
48
49 while True:
50     # 1. I2C Slave Path Validation
51     if slave.any():
52         b = slave.read(1)[0]
53         if 32 <= b <= 126:
54             slave_message += chr(b)
55         elif b == 10 or b == 13: # Newline/CR
56             if slave_message:
57                 print()
58                 print("PROTOCOL DETECTED : I2C SLAVE")
59                 print("MESSAGE :", slave_message)
60                 print()
61                 slave_message = ""
62
63     # 2. FPGA FIFO Polling
64     data = spi_read(64)
65     i = 0
66     while i < len(data):
67         b = data[i]
68
69         if b == 0x00:
70             i += 1
71             continue
72
73         if b == 0x01:
74             i2c_lines = ["START"]
75             i += 1
76             continue
77
78         if b == 0x02:
79             i2c_lines.append("STOP")
80             print()
81             print("PROTOCOL DETECTED : I2C")
82             for x in i2c_lines:
83                 print(x)
84             print()
85             i2c_lines = []
86             i += 1
87             continue
88
89         if b == 0x03:
90             if i + 2 >= len(data): break
91             addr = data[i + 1]
92             ack = data[i + 2]
93             rw = "READ" if (addr & 1) else "WRITE"
94             i2c_lines.append(
95                 "ADDR 0x%02X (%s) %s" % (addr >> 1, rw, "ACK" if ack == 0x05 else
96                 "NACK")
97             )
98             i += 3
99             continue

```



```

99
100     if b == 0x04:
101         if i + 2 >= len(data): break
102         val = data[i + 1]
103         ack = data[i + 2]
104         i2c_lines.append(
105             "DATA 0x%02X %s" % (val, "ACK" if ack == 0x05 else "NACK")
106         )
107         i += 3
108         continue
109
110     if b == 0xF1:
111         if i + 1 >= len(data): break
112         ch = data[i + 1]
113         if ch in (0x0D, 0x0A):
114             if uart_message:
115                 print()
116                 print("PROTOCOL DETECTED : UART")
117                 print("MESSAGE :", uart_message)
118                 print()
119                 uart_message = ""
120             elif 32 <= ch <= 126:
121                 uart_message += chr(ch)
122             i += 2
123             continue
124
125     if b == 0xD0:
126         spi_mosi_lines = []
127         spi_miso_lines = []
128         i += 1
129         continue
130
131     if b == 0xD1:
132         if i + 2 >= len(data): break
133         mosi_val = data[i + 1]
134         miso_val = data[i + 2]
135         spi_mosi_lines.append(mosi_val)
136         spi_miso_lines.append(miso_val)
137         i += 3
138         continue
139
140     if b == 0xD2:
141         print()
142         print("PROTOCOL DETECTED : SPI")
143         print()
144         print("START")
145         print()
146         print(f"BYTES : {len(spi_mosi_lines)}")
147         print()
148         mosi_str = " ".join(["%02X" % x for x in spi_mosi_lines])
149         miso_str = " ".join(["%02X" % x for x in spi_miso_lines])
150         print("MOSI : " + mosi_str)
151         print("MISO : " + miso_str)
152         print()
153         print("STOP")
154         print()
155         i += 1
156         continue
157
158     i += 1

```

Listing 10: RP2040 MicroPython Host Parser — main.py

9.3 Detailed Code Explanation

The host script is highly structured to accommodate the FPGA's pipelined architecture and the simultaneous nature of the protocols.

Initialization and FPGA Programming

The script begins by calling `shrike.flash()` to program the ForgeFPGA with the generated bitstream over I²C. Following this, it toggles the FPGA's reset pin (Pin 14) to ensure the internal logic initializes correctly. The hardware SPI interface and a software I²C slave instance (`RP2040_Slave`) are configured immediately afterward. The system is then fully armed to capture protocol events.

I²C Slave Validation Path

Inside the `while True` loop, the script first polls the `RP2040_Slave`. This serves as a secondary validation mechanism. If the ESP8266 generator sends an I²C string to address `0x48`, this code will physically receive it, accumulate the characters, and print it. If the FPGA's sniffer output (parsed later) matches this slave output, the design is verified as perfectly accurate.

FPGA FIFO Polling and Event Parsing

The host continuously reads 64-byte blocks from the FPGA's event FIFO via `spi_read(64)`. Because empty FIFO cycles return `0x00`, the script rapidly skips zero-bytes. When a non-zero marker byte is detected, the parser advances the index `i` appropriately:

- **I²C Markers (0x01–0x04):** Translates `0x01/0x02` into "START"/"STOP". For `0x03` (Address) and `0x04` (Data), it extracts the subsequent payload byte and the ACK/NACK status byte (`0x05/0x06`) and appends formatted human-readable strings to `i2c_lines`.
- **UART Marker (0xF1):** Extracts the next byte as an ASCII character, appending it to `uart_message` until a carriage return is detected, triggering a print.
- **SPI Markers (0xD0–0xD2):** Translates `0xD0/0xD2` into "START"/"STOP". For `0xD1` (Data), it extracts the subsequent parallel MOSI and MISO bytes from the next two positions, appending them to the packet buffers.

10 Hardware Interfacing and Pin Configurations

This section details the physical hardware connections and logical pin mappings used to interface the Vicharak Shrike Lite development platform with the external test traffic generator and host interfaces.

10.1 Go Configure IO Planner Mapping

The logical ports of the ForgeFPGA design are mapped to physical pins using the Go Configure I/O Planner. Table 5 lists the complete pin mappings, including directions and corresponding signals for clocking, reset, and protocol lines, covering monitoring inputs for UART, I²C SDA/SCL, and SPI MOSI/MISO/SCLK/CS.

S.No.	Function	Direction	Port
1	OSC_EN	Output	clk_en
2	OSC_CLK	Input	clk
3	GPIO18_IN [PIN 9]	Input	rst_n
4	GPIO16_OUT [PIN 7]	Output	led
5	GPIO16_OE [PIN 7]	Output	led_en
6	GPIO14_IN [PIN 5]	Input	spi_mon_miso
7	GPIO13_IN [PIN 4]	Input	spi_mon_mosi
8	GPIO11_IN [PIN 2]	Input	spi_mon_cs_n
9	GPIO10_IN [PIN 1]	Input	spi_mon_sclk
10	GPIO09_IN [PIN 24]	Input	i2c_scl
11	GPIO08_IN [PIN 23]	Input	i2c_sda
12	GPIO07_IN [PIN 20]	Input	uart_rx

Continued on next page

S.No.	Function	Direction	Port
13	GPIO06_OUT [PIN 19]	Output	spi_miso
14	GPIO06_OE [PIN 19]	Output	spi_miso_en
15	GPIO05_IN [PIN 18]	Input	spi_mosi
16	GPIO04_IN [PIN 17]	Input	spi_ss_n
17	GPIO03_IN [PIN 16]	Input	spi_sck

Table 5: Go Configure FPGA I/O Planner Pin Mapping

10.2 Physical Hardware Connections

S.No.	ESP8266 Pin	Shrike Lite / RP2040 Pin	Signal
UART Monitoring			
1	TX	F7 (FPGA)	uart_rx
I²C Monitoring			
2	D1 (GPIO5)	F9 (FPGA)	i2c_scl
3	D2 (GPIO4)	F8 (FPGA)	i2c_sda
4	D1 (GPIO5)	GPIO19 (RP2040)	i2c_scl (Slave)
5	D2 (GPIO4)	GPIO18 (RP2040)	i2c_sda (Slave)
SPI Monitoring			
6	D5 (GPIO14)	F10 (FPGA)	spi_mon_sclk
7	D8 (GPIO15)	F11 (FPGA)	spi_mon_cs_n
8	D7 (GPIO13)	F13 (FPGA)	spi_mon_mosi
9	D6 (GPIO12)	F14 (FPGA)	spi_mon_miso
Power			
10	GND	GND	Common Ground

Table 6: Physical Hardware Connections Table

For operational validation, the external ESP8266 test traffic generator was connected to the Vicharak Shrike Lite board. Table 5 defines the exact physical wiring connections established between the ESP8266 NodeMCU pins and the Shrike Lite FPGA input headers, including the additional SPI MOSI, MISO, SCLK, and CS monitoring connections.

11 Host Graphical User Interface (GUI)

11.1 Using the PyQt6 Host GUI

A pre-built Windows GUI executable is provided through the GitHub Releases section for users who wish to monitor protocol activity without running the terminal-based interface. This PyQt6-based application offers a highly intuitive, categorized, and visually appealing real-time dashboard for protocol analysis.

11.1.1 Download and Installation

1. Navigate to the **Releases** section of the project repository.
2. Download the latest release package: **Executable.zip**.
3. Extract the ZIP archive to any location on your computer.
4. Launch the **main.exe** application. No additional software dependencies or Python environments are required, as the application is completely standalone.

11.1.2 Connecting to the Analyzer

1. Connect the Vicharak Shrike Lite board to the host PC using a standard USB cable.
2. Ensure the protocol analyzer MicroPython firmware (`main.py`) is actively running on the RP2040.
3. Open the GUI application.
4. Select the appropriate **COM port** (corresponding to the RP2040's USB Serial interface) from the drop-down menu in the GUI.
5. Click **Connect**.

Once successfully connected, the GUI will automatically begin polling and receiving the decoded protocol packets from the FPGA-backed analyzer over the serial bus.

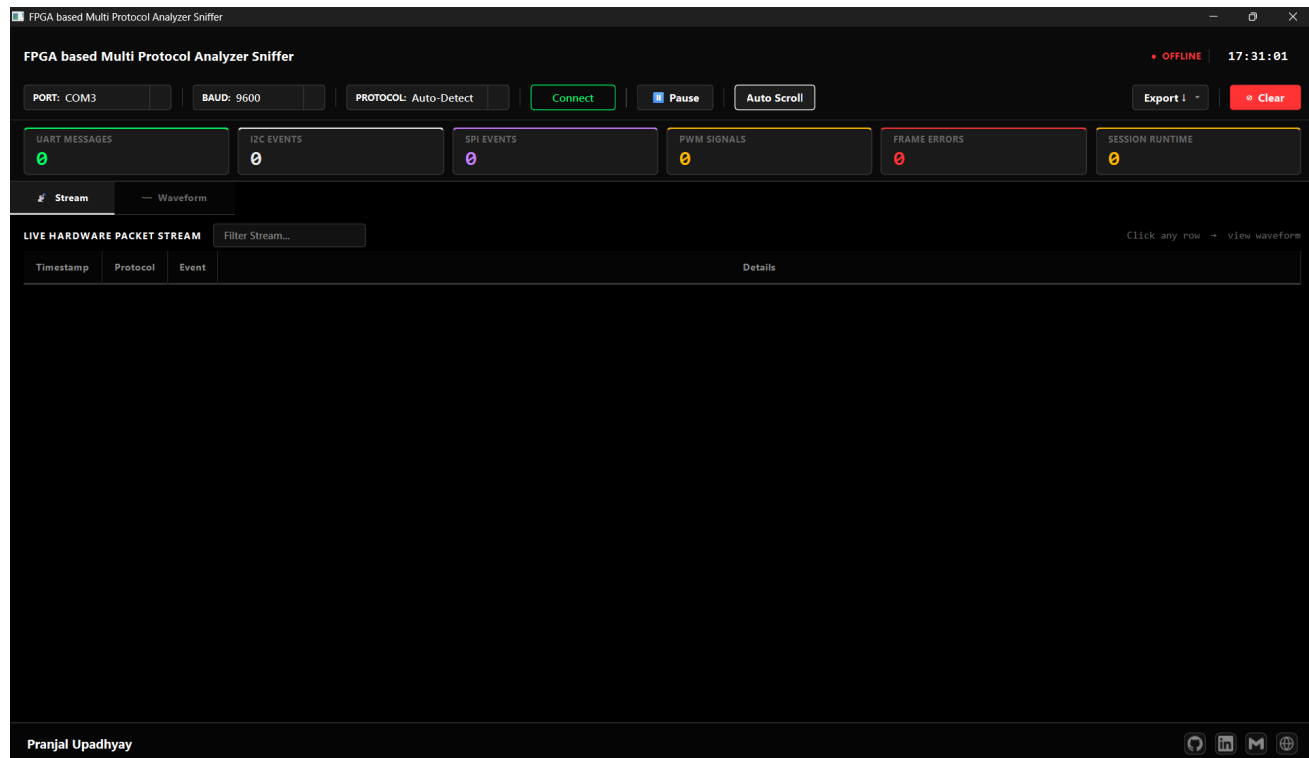


Figure 2: Main window of the PyQt6 Host GUI

11.2 Alternative: Thonny Shell Monitoring

Although the highly recommended and primary interface is the PyQt6 Host GUI, decoded protocol packets can also be monitored directly as raw text output through the Thonny IDE Shell. This is particularly useful for debugging or environments where the GUI executable cannot be run.

To use the fallback terminal mode:

1. Open the **Thonny IDE** on your host PC.
2. Connect to the Shrike board's MicroPython REPL.
3. Run the `main.py` analyzer firmware.
4. Observe the decoded UART, I²C, and SPI events streaming directly into the Shell window as formatted string blocks.

12 Experimental Results

This section presents the results obtained from functional validation of the complete Peripheral Analyzer Sniffer system. Experiments were conducted using the Vicharak Shrike Lite development board, with an ESP8266 NodeMCU module serving as the external traffic generator for UART, I²C, and SPI protocol tests. The FPGA performed all real-time signal acquisition, synchronization, and decoding, while the RP2040 forwarded the structured event packets to the host terminal over USB serial.

The experiments cover five key aspects of system behaviour: UART frame decoding under user-driven and programmatic traffic, I²C transaction decoding under message-based and random multi-byte scenarios, SPI loopback decoding under controlled full-duplex traffic, and a set of hardware-level observations related to metastability, FIFO arbitration, clock stretching compatibility, and SPI output-enable configuration. Together, these experiments validate the correctness of every subsystem in the analysis pipeline, from input synchronization through to host-side display.

12.1 UART Decoding Results

To evaluate the UART monitoring and decoding capabilities of the proposed Peripheral Analyzer Sniffer, a series of real-time communication experiments were performed. During testing, the ESP8266 served as the UART traffic source, while the FPGA continuously monitored the UART TX line connected to FPGA pin F7.

12.1.1 Example: User-Generated UART Message

In this experiment, a user manually entered the following message strings:

```
connection * 12309 % AS
peripheral ANALYZER sniffer
```

These characters were transmitted over the UART interface at the configured baud rate and simultaneously observed by the FPGA monitoring channel connected to pin F7.

The incoming UART signal was first synchronized using the FPGA's two-stage flip-flop synchronizer before being processed by the UART decoder module. The decoder detected each start bit, performed midpoint bit sampling, reconstructed the UART frames, extracted the transmitted bytes, and generated corresponding UART events.

The decoded event stream was buffered within the FPGA FIFO, transferred to the RP2040 through the SPI communication interface, and finally displayed on the host system using the PyQt6 Host GUI.

Figure 3 shows the decoded UART output observed directly on the Host GUI.

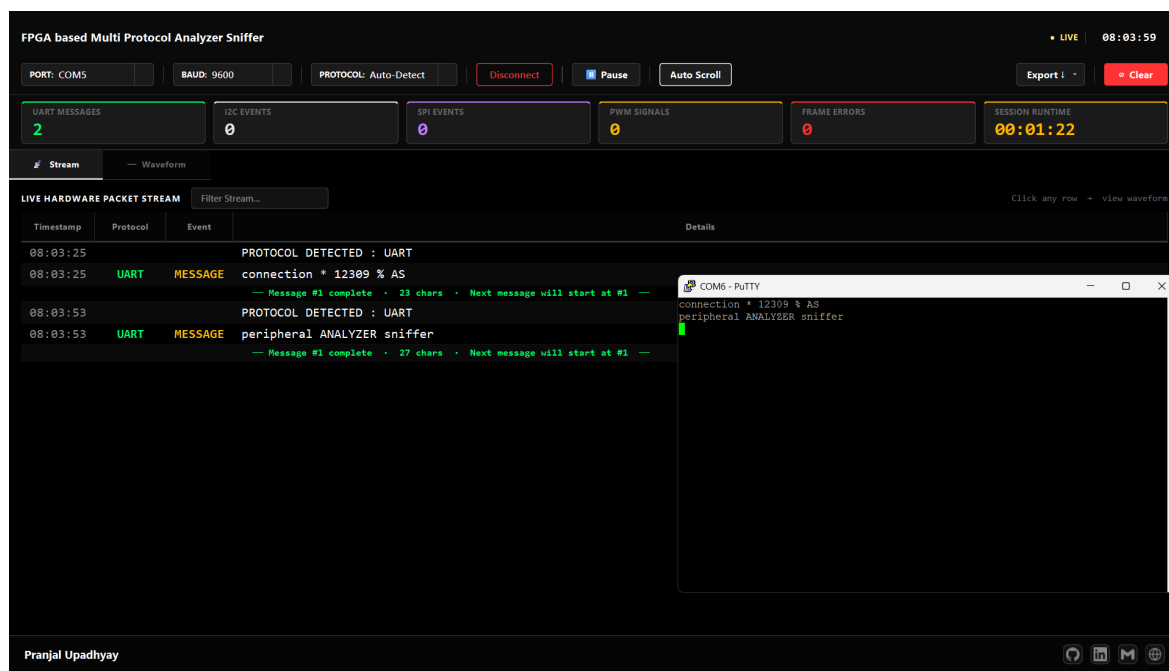


Figure 3: UART Example: FPGA-decoded UART messages displayed on the Host GUI.

As observed, the decoded output exactly matches the messages entered by the user. The analyzer correctly identified the communication protocol as UART, computed the payload length, and successfully reconstructed the complete message without character corruption or frame loss.

To physically verify the electrical properties and timing integrity of the transmitted frames alongside the GUI output, a hardware-level capture of the UART signal was also acquired.



Figure 4: Hardware-level UART frame capture confirming signal integrity and timing.

This experiment validates the functionality of the FPGA synchronization layer, UART decoder state machine, FIFO buffering subsystem, SPI transfer interface, and host-side display pipeline. The successful recovery of user-generated UART traffic demonstrates the system’s capability to perform real-time UART monitoring and decoding in practical debugging scenarios.

12.2 I²C Decoding Results

To evaluate the I²C monitoring subsystem, the ESP8266 was configured as an I²C master and a user manually entered the following message string through the terminal:

SERene 91

The characters were transmitted using slave address 0x48. The FPGA continuously monitored the SDA and SCL lines through monitoring pins F8 and F19, respectively.

12.2.1 Example: I²C Message Transmission

During operation, the FPGA I²C decoder successfully detected START conditions, slave addressing, data transfers, ACK responses, and STOP conditions. For each transaction, protocol events were generated, buffered within the FPGA FIFO, and transferred to the RP2040 through the SPI communication interface for display on the host terminal.

Figure 5 and Figure 6 show the decoded I²C output observed directly on the Host GUI.

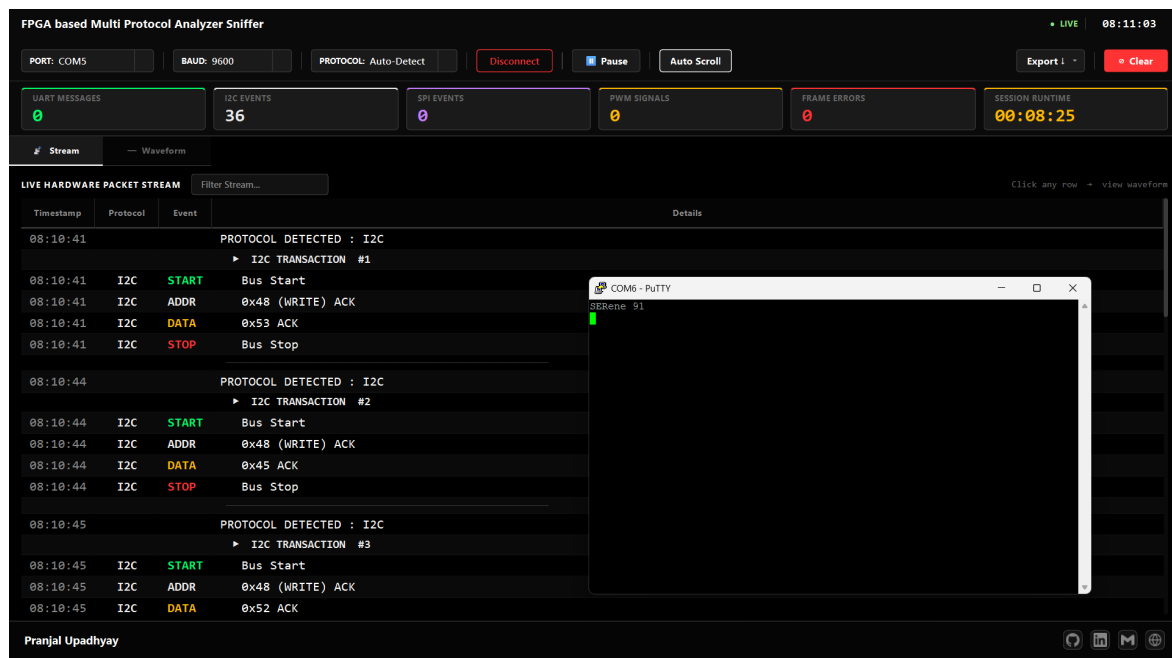


Figure 5: I²C Example: FPGA-decoded I²C transactions displayed on the Host GUI (Part 1).

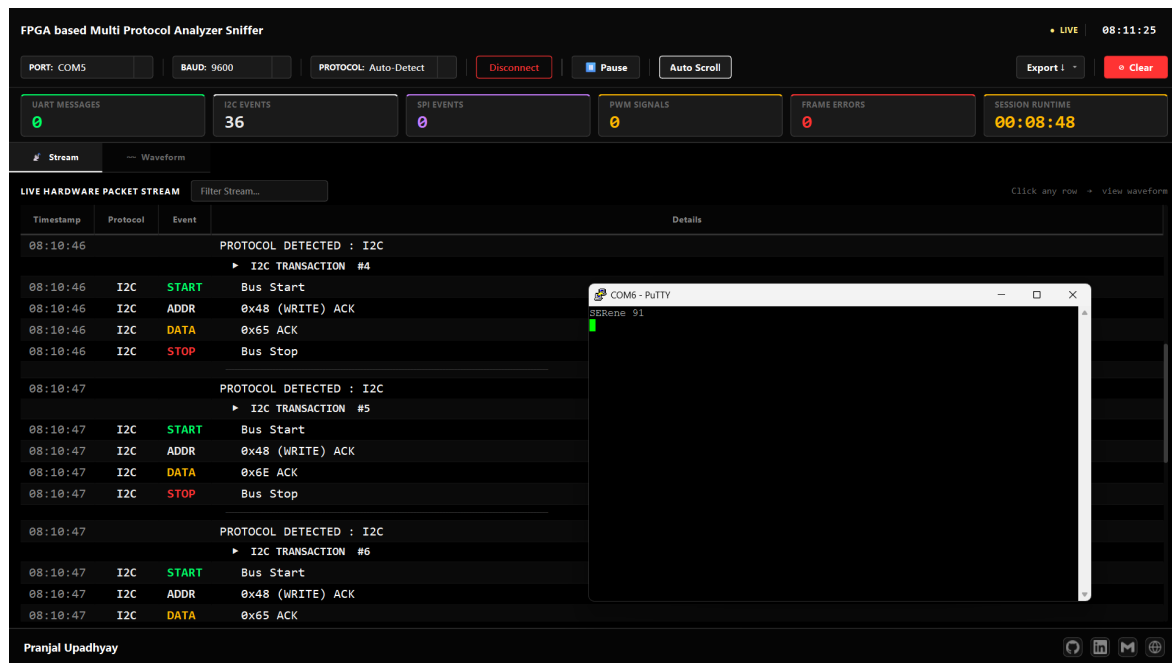


Figure 6: I²C Example: FPGA-decoded I²C transactions displayed on the Host GUI (Part 2).

For every transaction, the FPGA correctly identified the protocol as I²C and successfully decoded the START condition, slave address, transfer direction, data byte, acknowledgment status, and STOP condition. The slave address remained consistently decoded as 0x48, while all transmitted data bytes matched the expected ASCII encoding of the SERene 91 message.

To physically verify the electrical properties and timing integrity of the transmitted frames alongside the GUI output, a hardware-level capture of the I²C signal was also acquired.

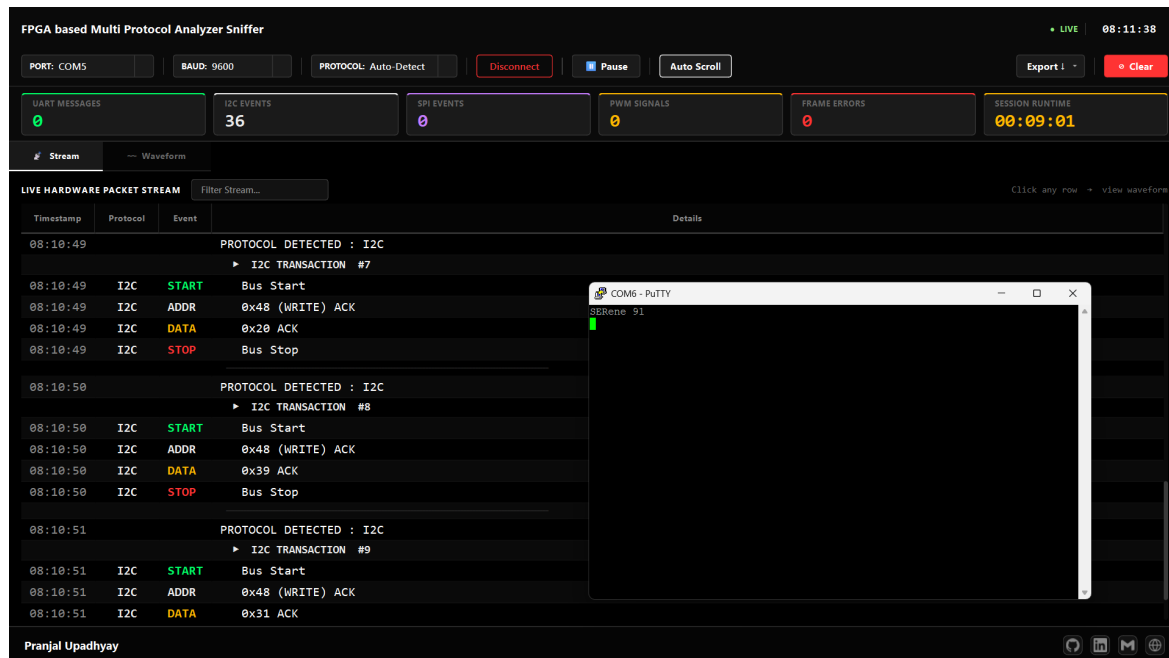


Figure 7: Hardware-level I²C frame capture confirming signal integrity and timing.

The successful reconstruction of the complete message demonstrates the correctness of the SDA/SCL synchronization circuitry, I²C decoder state machine, address extraction logic, data-byte reconstruction logic, FIFO buffering subsystem, SPI communication interface, and host-side reporting pipeline.

These results confirm that the proposed Peripheral Analyzer Sniffer can reliably monitor and decode real-time I²C communication entirely in FPGA hardware while accurately presenting human-readable protocol information on the host terminal.

12.3 SPI Decoding Results

To evaluate the SPI monitoring subsystem, the ESP8266 was configured as an SPI master and programmed to transmit the following test strings over the SPI bus:

```
work @ 100
* 5339 033 peculiar
```

The FPGA continuously monitored the externally routed CS, SCLK, MOSI, and MISO lines through the dedicated SPI decoder monitoring pins.

12.3.1 Decoded SPI Output

After synchronization through the two-stage flip-flop synchronizers dedicated to CS, SCLK, MOSI, and MISO, the FPGA SPI decoder detected each CS falling edge, sampled MOSI and MISO simultaneously on every active SCLK edge, reconstructed the full-duplex byte pair, and stored them in the common FIFO event buffer. The captured SPI frames were retrieved from the FIFO and displayed on the Host GUI.

Figure 8 shows the decoded SPI output observed directly on the Host GUI.

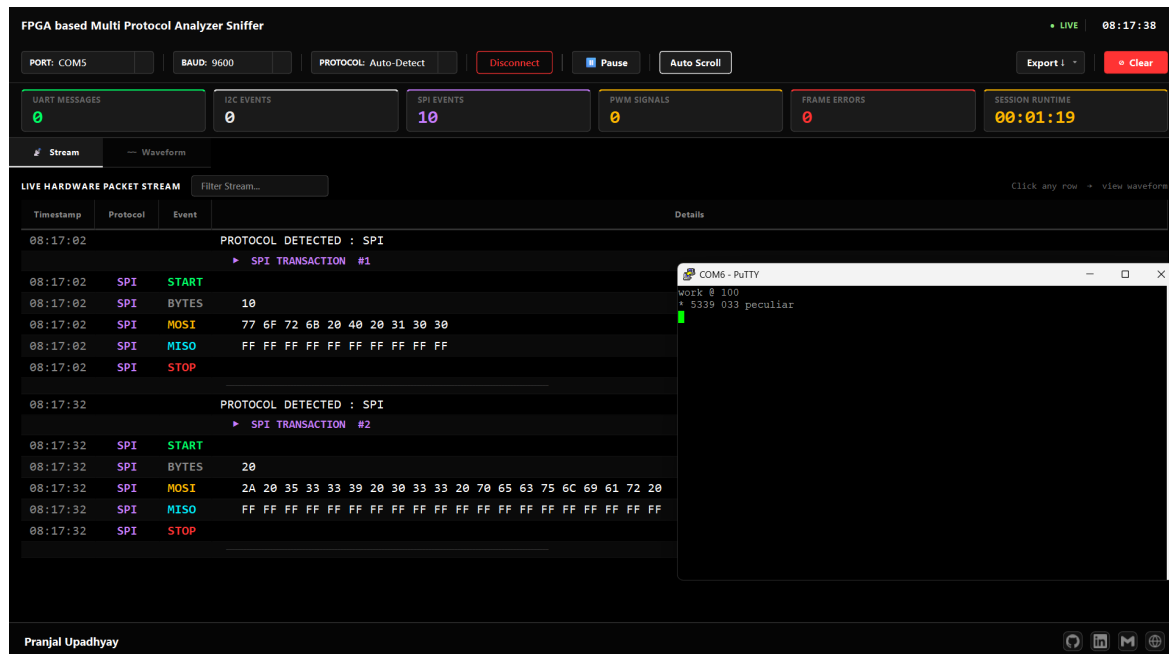


Figure 8: SPI Example: FPGA-decoded SPI transactions displayed on the Host GUI.

The decoded output exactly matches the strings transmitted by the SPI master. Each transaction was correctly bracketed by an SPI START event on the CS falling edge and an SPI STOP event on the CS rising edge, with no partial or truncated frames observed across the test runs.

This experiment validates the correctness of the CS edge-detection logic, the SCLK sampling-edge selection logic, the parallel MOSI/MISO shift registers, the byte-reconstruction counter, and the buffering of captured SPI frames in the synchronous FIFO buffer for asynchronous retrieval by the host.

12.4 Metastability at the FPGA Input Boundary

The most fundamental hardware challenge encountered during early testing was the absence of the two-stage synchronizer. Initial prototype builds routed the ESP8266 output signals directly to the decoder state machine inputs without passing through synchronizers. The resulting behaviour was diagnostic: the UART byte stream exhibited occasional single-bit errors (random bit flips in otherwise correct bytes), the I²C decoder produced sporadic spurious NACK detections during long transactions, and the early SPI decoder occasionally mis-shifted a single bit within a byte during sustained loopback traffic. All three failure modes are precisely consistent with metastability-induced flip-flop state corruption: a metastable state propagates through the FSM's combinational next-state logic, causing an incorrect state transition that corrupts the current bit value.

Inserting the `two_stage_sync` module at all FPGA input boundaries — UART RX, SDA, SCL, and SPI CS, SCLK, MOSI, and MISO — immediately and completely eliminated all three classes of errors across all subsequent tests. This empirical result provides direct experimental validation of the theoretical synchronizer analysis presented in Section 3.5.

12.5 Simultaneous FIFO Write Arbitration

The UART decoder (`byte_ready`) and the I²C decoder (`event_valid`) can both assert their write-enable outputs in the same clock cycle during concurrent mixed-protocol operation. Since both decoders share a single FIFO input port, simultaneous write enables would cause a write collision in which only one event's data is stored. The top-level FPGA module implements a fixed-priority combinational arbitration circuit: when both decoders request a write in the same clock cycle, the I²C decoder is given priority, and the UART byte is queued on the next available cycle. The SPI decoder also participates in this arbitration, ensuring that all three protocols can share the common FIFO without data corruption.

12.6 I²C Clock Stretching Compatibility

During initial I²C testing with a real sensor device (rather than the ESP8266 simulator), the sensor implemented clock stretching by holding SCL LOW during internal ADC conversion. Because the I²C decoder samples SDA exclusively on SCL rising edges (not on any fixed clock count), clock stretching is inherently tolerated: the decoder simply remains in its bit-capture state indefinitely until SCL is released by the slave and rises again. No special handling was required; the architecture naturally accommodates this standard I²C feature.

12.7 SPI MISO Output-Enable Configuration

A critical challenge during board integration was managing the SPI MISO line's driver state. The SPI MISO output from the FPGA is routed to a shared bus connected to the host controller. If the FPGA's MISO output buffer remained active when the target select line (`spi_ss_n`) was high, it would cause bus contention, preventing other devices from utilizing the SPI bus. To resolve this, the FPGA top-level module maps the SPI Target block's output-enable signal (`o_miso_oe`) to drive the physical output enable pin `spi_miso_en`. This ensures that the MISO pin is driven only when the select line is low, and goes into a high-impedance state when the target is deselected.

12.8 SPI Clock Domain Crossing Synchronization

Because the SPI interface clock (`i_sck`) is generated by an external host and operates asynchronously to the FPGA system clock (`clk`), a clock-domain crossing (CDC) exists. If the FPGA sampled the raw SCK line directly, setup/hold timing violations would occur, leading to register metastability and corrupt transmission pointer states. This was resolved by passing both `i_ss_n` and `i_sck` through 3-bit synchronization shift registers (`r_ss_n_sync` and `r_sck_sync`) clocked by the 50 MHz internal clock. All clock edge detection and bit counting are performed on these synchronized signals, ensuring error-free data transfer. The same synchronization discipline is applied independently to the SPI *decoder* block's CS, SCLK, MOSI, and MISO monitoring inputs, which use the standard two-stage synchronizer rather than the 3-bit shift register used for the host-link SPI target.

13 Future Scope and Roadmap

13.1 Near-Term FPGA Protocol Additions

13.1.1 CAN Decoder

The Controller Area Network (CAN) protocol uses a differential two-wire bus (`CAN_H`, `CAN_L`) with bit-stuffing, arbitration, and CRC fields. A CAN decoder extension would require:

- A differential receiver front-end (external to the FPGA fabric) to convert the `CAN_H`/`CAN_L` differential signal to a single-ended digital input.
- A bit-timing FSM that performs bit-stuffing removal, arbitration field decoding, and CRC verification.
- A new event type code, `EV_CAN_FRAME`, carrying the CAN identifier, data length code, and up to 8 data bytes.

13.1.2 LIN Decoder

The Local Interconnect Network (LIN) protocol is a low-cost, single-wire, UART-derived bus commonly used in automotive body electronics. Because LIN's physical-layer bit timing is electrically similar to UART, a LIN decoder could reuse much of the existing UART receiver core, extended with:

- Break-field detection (an extended LOW period that the current UART decoder explicitly does not handle, as noted in Section 3.1.4).
- Sync-byte and Protected Identifier (PID) field parsing.
- Checksum verification against the LIN 2.x checksum algorithm.

13.1.3 Automatic UART Baud Rate Detection

An FPGA block that eliminates the need for hardcoded `BIT_PERIOD`:

- On the first start-bit falling edge, a free-running counter begins.
- On the next rising edge of the UART signal (the end of the start bit), the counter value is latched and right-shifted by one to produce `HALF_PERIOD`.
- The UART decoder's timing parameters are loaded from these measured values before the first data bit is decoded.
- A confidence window check validates that the measured period falls within the range of a standard baud rate ($\pm 3\%$).

13.1.4 1-Wire Protocol Decoder

The Dallas/Maxim 1-Wire protocol is used by DS18B20 temperature sensors and iButton devices. It uses a single bidirectional signal line with a complex reset-pulse / presence-pulse handshake followed by bit-by-bit time-slot encoding. A 1-Wire decoder in the FPGA would require one additional synchronized input channel and an FSM implementing the reset-detect, bit-sample, and byte-assembly sequence.

13.2 FPGA Architecture Improvements

- **Multi-Entry Priority Arbiter:** Replace the simple fixed-priority UART/I²C write arbiter with a round-robin arbiter. This enables truly simultaneous capture of both protocols, while SPI remains decoupled via the shared FIFO buffer.
- **Deeper FIFO with Overflow Event:** Increase FIFO depth to 128 entries ($128 \times 16 \text{ bits} = 2048 \text{ bits} = 256 \text{ bytes}$). Add an EV_OVERFLOW event type that the FIFO emits automatically when a write is rejected due to full status, allowing the host display to annotate missed-event gaps in the output log.
- **Hardware Configurable Filter Block:** Implement a 7-bit address comparison register (loaded via a configuration command packet from the host) that selectively drops I²C events for non-matching addresses before they enter the FIFO. This dramatically reduces USB bandwidth consumption on high-traffic I²C buses with many devices.
- **32-bit Event Timestamp Counter:** A free-running 32-bit counter clocked at the FPGA system clock provides a timestamp with 100 ns resolution. The counter value is appended to each event before FIFO entry, requiring the FIFO word width to be extended from 16 to 48 bits and the packet format to be extended by 4 bytes.
- **Configurable SPI Word Width:** Extend the SPI decoder to support 16-bit and 32-bit word widths via a runtime-configurable bit-count parameter, rather than the fixed 8-bit width used in this project's validation.

13.3 Host-Side and Firmware Improvements

- **Python GUI Application:** A cross-platform Python application (using PyQt5 or Tkinter) that connects to the host SPI port, parses the text output stream, and renders decoded transactions in a scrollable timeline view with colour-coded protocol lanes (similar to Saleae Logic's protocol decoder display). Export to CSV and VCD (Value Change Dump) formats would enable post-capture analysis in standard EDA tools.
- **Waveform Viewer:** An integrated waveform viewer within the GUI application that reconstructs approximate signal timing from decoded event timestamps, presenting UART, I²C, and SPI activity on a shared, zoomable timeline alongside the decoded byte values.
- **Session Manager:** A session management layer in the GUI that allows the user to save, load, name, and compare multiple capture sessions, supporting longer-term debugging workflows that span multiple power cycles or test scenarios.
- **Packet Inspector:** A detailed packet inspector view that allows the user to click on any decoded event and see its full raw byte sequence, timestamp, protocol classification, and any associated error or warning flags, in a manner similar to commercial protocol analyzer software.
- **AI-Assisted Protocol Debugging:** An optional analysis layer that applies pattern recognition across captured event sequences to flag anomalies (unexpected NACKs, malformed SPI frames, unusual UART framing errors) and suggest likely root causes, reducing the manual effort required to interpret large capture logs.
- **I²C Device Address Database:** A local lookup table cross-referencing decoded I²C device addresses against a curated database of known components: 0x48 = TMP102 (Texas Instruments), 0x68 = MPU-6050 (InvenSense), 0x3C = SSD1306 OLED (Solomon Systech), 0x50–0x57 = AT24C EEPROM (Microchip). Matching entries would append the device name to the address line in the terminal output, instantly identifying unknown peripherals on a new hardware platform.
- **Configurable Baud Rate via USB Command Interface:** Implement a command packet format in the host-to-FPGA direction (using the SPI MOSI line, or a dedicated SPI command frame channel) allowing the PC application to send configuration commands to the FPGA (set baud rate, set I²C filter address, enable/disable protocols) without reprogramming the bitstream.

- **Trigger Capture Mode:** Add a “trigger on first matching event” mode: the FPGA begins filling the FIFO but does not assert the host-interface transmit enable until a user-specified trigger pattern (address match, data value match, STOP condition) is detected. This allows pre-trigger and post-trigger event capture, analogous to an oscilloscope trigger system.

14 Conclusion

This work presented the design, implementation, and experimental validation of a hardware-accelerated **Multi-Protocol Peripheral Analyzer Sniffer** implemented on the Vicharak Shrike Lite platform. The developed system successfully performs real-time monitoring and decoding of UART, I²C, and SPI communication protocols using dedicated FPGA hardware, thereby eliminating the need for protocol decoding within the host microcontroller.

The proposed architecture employs the Renesas SLG47910 ForgeFPGA as the primary protocol-analysis engine, where asynchronous signal synchronization, metastability mitigation, protocol detection, frame reconstruction, and event generation are performed entirely in hardware. Dedicated decoder modules were implemented for UART, I²C, and SPI communication, while a FIFO-based event transport mechanism enables reliable transfer of decoded protocol information to the RP2040. The RP2040 operates solely as a communication interface, responsible for packet acquisition, formatting, and host-side presentation through USB serial communication and the graphical user interface.

Experimental validation confirmed the correct operation of all implemented protocol decoders. The UART decoder successfully reconstructed asynchronous serial data streams, the I²C decoder correctly identified START and STOP conditions, addresses, data bytes, and ACK/NACK responses, and the SPI decoder accurately reconstructed MOSI and MISO data transfers under controlled test conditions. These results demonstrate the effectiveness of the proposed FPGA-based approach for real-time protocol monitoring and analysis.

The project further illustrates the practical application of key digital design concepts including clock-domain crossing protection, two-stage synchronizers, finite-state-machine-based protocol decoding, event buffering, and FPGA-to-MCU communication. The resulting system provides a compact and cost-effective platform for embedded-system debugging, protocol validation, and engineering education.

In conclusion, the developed Multi-Protocol Peripheral Analyzer Sniffer successfully achieves its stated objectives and demonstrates the viability of hardware-based protocol analysis using a co-integrated FPGA and microcontroller architecture. The modular design also provides a foundation for future enhancements and protocol-expansion capabilities.

References

- [1] Vicharak. *Shrike Lite GitHub Repository*. Available at <https://github.com/vicharak-in/shrike>
- [2] Circuit Digest. *Electronics Engineering News, Projects & Tutorials*. Available at <https://circuitdigest.com>
- [3] Raspberry Pi Ltd. *RP2040 Microcontroller Documentation*. Available at <https://www.raspberrypi.com/documentation/microcontrollers/rp2040.html>
- [4] Renesas Electronics. *ForgeFPGA SLG47910 Device Datasheet & Documentation*. Available at <https://www.renesas.com/forgefpga>

Project Resources

Source Code Repository:

The complete Verilog source code, MicroPython firmware, Host GUI application, and project documentation are available on GitHub:

<https://github.com/upadhyaypranjal/FPGA-based-Multi-Protocol-Analyzer-Sniffer>

Video Demonstration:

A full video demonstration of the Peripheral Analyzer Sniffer system in action can be viewed here:

[Google Drive: Project Video Demo](#)